MAS ALL

ROUNDER

MAS ALL

FOR MORE VU DATA VISIT MY website:

https://masallrounder.blogspot.com/

ROUNDER

CS402 Midterm Preparation Lecture wise by

PIN2 and MUHAMMAD

(MAS All Rounder)

Lecture N0. 1

Reading Material

Introduction to Computer Theory Chapter 2

Summary
Introduction to the course title, Formal and In-formal languages, Alphabets, Strings, Null string, Words, Valid and In-valid alphabets, length of a string, Reverse of a string, Defining languages, Descriptive definition of languages, EQUAL, EVEN-EVEN, INTEGER, EVEN, $\{ a^n b^n \}$, $\{ a^n b^n a^n \}$, factorial, FACTORIAL, DOUBLEFACTORIAL, SQUARE, DOUBLESQUARE, PRIME, PALINDROME.

What does automata mean?
It is the plural of automaton, and it means "something that works automatically"

Introduction to languages

There are two types of languages

- Formal Languages (Syntactic languages)
- Informal Languages (Semantic languages)

MAS ALL ROUNDER

Alphabets
Definition
A finite non-empty set of symbols (called letters), is called an alphabet. It is denoted by Σ (Greek letter sigma).

Example

Σ = {a,b}

Σ = {0,1} (*important as this is the language which the computer understands*.)

Σ = {i,j,k}

Note Certain version of language ALGOL has 113 letters.

Σ (alphabet) includes letters, digits and a variety of operators including sequential operators such as GOTO and IF

Strings

Definition
Concatenation of finite number of letters from the alphabet is called a string.

Example

If Σ = {a,b} then

a, abab, aaabb, abababababababababab

Note

Empty string or null string

Sometimes a string with no symbol at all is used, denoted by (Small Greek letter Lambda) λ or (Capital Greek letter Lambda) Λ, is called an empty string or null string.

The capital lambda will mostly be used to denote the empty string, in further discussion.

Words

Definition

Words are strings belonging to some language.

Example

If Σ= {x} then a language L can be defined as

$L=\{x^n : n=1,2,3,.....\}$ or L={x,xx,xxx,....}

Here x,xx,… are the words of L

Note

All words are strings, but not all strings are words.

Valid/In-valid alphabets

While defining an alphabet, an alphabet may contain letters consisting of group of symbols for example Σ_1= {B, aB, bab, d}.

Now consider an alphabet

Σ_2= {B, Ba, bab, d} and a string BababB.

This string can be tokenized in two different ways

(Ba), (bab), (B)

(B), (abab), (B)

Which shows that the second group cannot be identified as a string,
defined over Σ = {a, b}.

As when this string is scanned by the compiler (Lexical Analyzer), first symbol B is identified as a letter belonging to Σ, while for the second letter the lexical analyzer would not be able to identify, so while defining an alphabet it should be kept in mind that ambiguity should not be created.

Remarks

While defining an alphabet of letters consisting of more than one symbols, no letter should be started with the letter of the same alphabet *i.e.* one letter should not be the prefix of another. However, a letter may be ended in a letter of same alphabet.

Conclusion

Σ_1= {B, aB, bab, d}

Σ_2= {B, Ba, bab, d}

Σ_1 is a valid alphabet while Σ_2 is an in-valid alphabet.

Length of Strings

Definition

The length of string s, denoted by |s|, is the number of letters in the string.

Example

Σ={a,b}

s=ababa

|s|=5

Example

Σ= {B, aB, bab, d}

s=BaBbabBd

Tokenizing=(B), (aB), (bab), (B), (d)

|s|=5

Reverse of a String

Definition

The reverse of a string s denoted by Rev(s) or s^r, is obtained by writing the letters of s in reverse order.

Example

If s=abc is a string defined over

Σ={a,b,c} then Rev(s) or s^r = cba

Example

Σ= {B, aB, bab, d}

s=BaBbabBd

Rev(s)=dBbabaBB

Defining Languages

The languages can be defined in different ways , such as Descriptive definition, Recursive definition, using Regular Expressions(RE) and using Finite Automaton(FA) etc.

Descriptive definition of language

The language is defined, describing the conditions imposed on its words.

Example

The language L of strings of odd length, defined over Σ={a}, can be written as L={a, aaa, aaaaa,…..}

Example

The language L of strings that does not start with a, defined over Σ ={a,b,c}, can be written as L ={Λ, b, c, ba, bb, bc, ca, cb, cc, …}

Example

The language L of strings of length 2, defined over Σ ={0,1,2}, can be written as L={00, 01, 02,10, 11,12,20,21,22}

Example

The language L of strings ending in 0, defined over Σ ={0,1}, can be written as

L={0,00,10,000,010,100,110,…}

Example

The language **EQUAL**, of strings with number of a's equal to number of b's, defined over Σ={a,b}, can be written as

{Λ ,ab,ba,aabb,abab,baba,abba,…}

Example

The language **EVEN-EVEN**, of strings with even number of a's and even number of b's, defined over Σ={a,b}, can be written as

{Λ, aa, bb, aaaa,aabb,abab, abba, baab, baba, bbaa, bbbb,…}

Example

The language **INTEGER**, of strings defined over Σ={-,0,1,2,3,4,5,6,7,8,9}, can be written as INTEGER = {…,-2,-1,0,1,2,…}

MAS ALL ROUNDER

Example

The language **EVEN**, of stings defined over Σ={-,0,1,2,3,4,5,6,7,8,9}, can be written as EVEN = { …,-4,-2,0,2,4,…}

Example

The language $\{a^n b^n\}$, of strings defined over Σ={a,b}, as

$\{a^n b^n : n=1,2,3,\ldots\}$, can be written as

{ab, aabb, aaabbb,aaaabbbb,…}

Example

The language $\{a^n b^n a^n\}$, of strings defined over Σ={a,b}, as

$\{a^n b^n a^n: n=1,2,3,\ldots\}$, can be written as

{aba, aabbaa, aaabbbaaa,aaaabbbbaaaa,…}

Example

The language **factorial**, of strings defined over Σ={0,1,2,3,4,5,6,7,8,9} *i.e.*

{1,2,6,24,120,…}

Example

The language **FACTORIAL**, of strings defined over Σ={a}, as

$\{a^{n!} : n=1,2,3,\ldots\}$, can be written as

{a,aa,aaaaaa,…}. It is to be noted that the language FACTORIAL can be defined over any single letter alphabet.

Example

The language **DOUBLEFACTORIAL**, of strings defined over Σ={a, b}, as

$\{a^{n!}b^{n!} : n=1,2,3,\ldots\}$, can be written as

{ab, aabb, aaaaaabbbbbb,…}

Example

The language **SQUARE**, of strings defined over Σ={a}, as

$\{a^{n2} : n=1,2,3,\ldots\}$, can be written as

{a, aaaa, aaaaaaaaa,…}

Example

The language **DOUBLESQUARE**, of strings defined over Σ={a,b}, as

$\{a^{n^2} b^{n^2} : n=1,2,3,\ldots\}$, can be written as

{ab, aaaabbbb, aaaaaaaaabbbbbbbbb,…}

Example

The language **PRIME**, of strings defined over Σ={a}, as

$\{a^p : p \text{ is prime}\}$, can be written as

{aa,aaa,aaaaa,aaaaaaa,aaaaaaaaaaa…}

An Important language

PALINDROME

The language consisting of Λ and the strings s defined over Σ such that Rev(s)=s. It is to be denoted that the words of PALINDROME are called palindromes.

Example

For Σ={a,b},

PALINDROME={Λ , a, b, aa, bb, aaa, aba, bab, bbb, ...}

Remark

There are as many palindromes of length 2n as there are of length 2n-1. To prove the above remark, the following is to be noted:

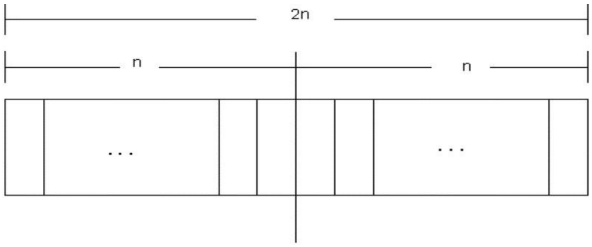
Note

Number of strings of length 'm' defined over alphabet of 'n' letters is n^m.

Examples

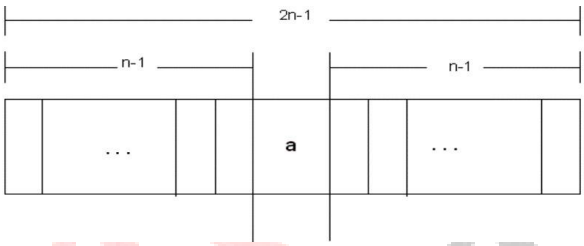
The language of strings of length 2, defined over Σ={a,b} is L={aa, ab, ba, bb} *i.e.* number of strings = 2^2 The language of strings of length 3, defined over Σ={a,b} is L={aaa, aab, aba, baa, abb, bab, bba, bbb} *i.e.* number of strings = 2^3

To calculate the number of palindromes of length(2n), consider the following diagram,



which shows that there are as many palindromes of length 2n as there are the strings of length n *i.e.* the required number of palindromes are 2^n.

To calculate the number of palindromes of length (2n-1) with 'a' as the middle letter, consider the following diagram,



which shows that there are as many palindromes of length 2n-1 as there are the strings of length n-1 *i.e.* the required number of palindromes are 2^{n-1}.

Similarly the number of palindromes of length 2n-1, with ' b ' as middle letter, will be 2^{n-1} as well. Hence the total number of palindromes of length 2n-1 will be $2^{n-1} + 2^{n-1} = 2\,(2^{n-1}) = 2^n$.

Theory of Automata

Lecture N0 **Reading Material** **2**

Introduction to Computer Theory Chapter **2**

Summary

Kleene Star Closure, Plus operation, recursive definition of languages, INTEGER, EVEN, factorial, PALINDROME, {$\mathbf{a^n b^n}$}, languages of strings (i) ending in a, (ii) beginning and ending in same letters, (iii) containing aa or bb (iv) containing exactly one a

Kleene Star Closure

Given Σ, then the Kleene Star Closure of the alphabet Σ, denoted by Σ^*, is the collection of all strings defined over Σ, including Λ.

It is to be noted that Kleene Star Closure can be defined over any set of strings.

Examples

If Σ = {x}

Then Σ^* = {Λ, x, xx, xxx, xxxx, ….}

If Σ = {0,1}

Then Σ^* = {Λ, 0, 1, 00, 01, 10, 11, ….}

If Σ = {aaB, c}

Then Σ^* = {Λ, aaB, c, aaBaaB, aaBc, caaB, cc, ….}

Note

Languages generated by Kleene Star Closure of set of strings, are infinite languages. (By infinite language, it is supposed that the language contains infinite many words, each of finite length).

PLUS Operation ($^+$)

Plus Operation is same as Kleene Star Closure except that it does not generate Λ (null string), automatically.

Example

If Σ = {0,1}

Then Σ^+ = {0, 1, 00, 01, 10, 11, ….}

If Σ = {aab, c}

Then Σ^+ = {aab, c, aabaab, aabc, caab, cc, ….}

Remark

It is to be noted that Kleene Star can also be operated on any string *i.e.* a^* can be considered to be all possible strings defined over {a}, which shows that a^* generates Λ, a, aa, aaa, …

It may also be noted that a^+ can be considered to be all possible non empty strings defined over {a}, which shows that a^+ generates a, aa, aaa, aaaa, …

Recursive definition of languages

The following three steps are used in recursive definition Some basic words are specified in the language.

Rules for constructing more words are defined in the language.

No strings except those constructed in above, are allowed to be in the language.

Examples

Defining language of INTEGER

Step 1: 1 is in **INTEGER**.

Step 2: If x is in **INTEGER** then x+1 and x-1 are also in **INTEGER**.

Step 3: No strings except those constructed in above, are allowed to be in **INTEGER**.

Defining language of EVEN

Step 1: 2 is in **EVEN**.

Step 2: If x is in **EVEN** then x+2 and x-2 are also in **EVEN**.

Step 3: No strings except those constructed in above, are allowed to be in **EVEN**.

Defining the language factorial

Step 1: As 0!=1, so 1 is in **factorial**.

Step 2: n!=n*(n-1)! is in **factorial**.

Step 3: No strings except those constructed in above, are allowed to be in **factorial**.

Defining the language PALINDROME, defined over $\Sigma = \{a,b\}$

Step 1: a and b are in **PALINDROME**

Step 2: if x is palindrome, then s(x)Rev(s) and xx will also be palindrome, where s belongs to Σ^*

Step 3: No strings except those constructed in above, are allowed to be in palindrome

Defining the language $\{a^n b^n\}$, n=1,2,3,… , of strings defined over $\Sigma=\{a,b\}$

Step 1: ab is in $\{a^n b^n\}$

Step 2: if x is in $\{a^n b^n\}$, then axb is in $\{a^n b^n\}$

Step 3: No strings except those constructed in above, are allowed to be in $\{a^n b^n\}$

Defining the language L, of strings ending in a , defined over $\Sigma=\{a,b\}$

Step 1: a is in L

Step 2: if x is in L then s(x) is also in **L,** where s belongs to Σ^*

Step 3: No strings except those constructed in above, are allowed to be in **L**

Defining the language L, of strings beginning and ending in same letters , defined over $\Sigma=\{a, b\}$

Step 1: a and b are in L

Step 2: (a)s(a) and (b)s(b) are also in **L,** where s belongs to Σ^*

Step 3: No strings except those constructed in above, are allowed to be in **L**

Defining the language L, of strings containing aa or bb , defined over $\Sigma=\{a, b\}$

Step 1: aa and bb are in L

Step 2: s(aa)s and s(bb)s are also in **L,** where s belongs to Σ^*

Step 3: No strings except those constructed in above, are allowed to be in **L**

Defining the language L, of strings containing exactly one a, defined over Σ={a, b}

Step 1: a is in L

Step 2: s(a)s is also in **L,** where s belongs to b^*

Step 3: No strings except those constructed in above, are allowed to be in **L**

Theory of Automata

Lecture N0. 3

Reading Material

Introduction to Computer Theory Chapter 4

Summary

RE, Recursive definition of RE, defining languages by RE, { x}*, { x}+, {a+b}*, Language of strings having exactly one a, Language of strings of even length, Language of strings of odd length, RE defines unique language (as Remark), Language of strings having at least one a, Language of strings having at least one a and one b, Language of strings starting with aa and ending in bb, Language of strings starting with and ending in different letters.

Regular Expression

As discussed earlier that a^* generates Λ, a, aa, aaa, … and a^+ generates a, aa, aaa, aaaa, …, so the language L_1 = {Λ, a, aa, aaa, …} and L_2 = {a, aa, aaa, aaaa, …} can simply be expressed by a^* and a^+, respectively. a^* and a^+ are called the regular expressions (RE) for L_1 and L_2 respectively.

Note a^+, aa^* and a^*a generate L_2.

Recursive definition of Regular Expression(RE)

Step 1: Every letter of Σ including Λ is a regular expression.

Step 2: If r_1 and r_2 are regular expressions then

(r_1)

r_1 r_2 r_1 + r_2 and r_1* are also regular expressions.

Step 3: Nothing else is a regular expression.

Method 3 (Regular Expressions)

Consider the language L={Λ, x, xx, xxx,…} of strings, defined over Σ = {x}.

We can write this language as the Kleene star closure of alphabet Σ or $L=\Sigma^*=\{x\}^*$.

This language can also be expressed by the regular expression x^*.

Similarly the language L={x, xx, xxx,…}, defined over Σ = {x}, can be expressed by the regular expression x^+. Now consider another language L, consisting of all possible strings, defined over Σ = {a, b}. This language can also be expressed by the regular expression (a + b)*.

Now consider another language L, of strings having exactly one a, defined over Σ = {a, b}, then it's regular expression may be b*ab*.

Now consider another language L, of even length, defined over Σ = {a, b}, then it's regular expression may be ((a+b)(a+b))*.

Now consider another language L, of odd length, defined over Σ = {a, b}, then it's regular expression may be (a+b)((a+b)(a+b))* or ((a+b)(a+b))*(a+b).

Remark

It may be noted that a language may be expressed by more than one regular expression, while given a regular expression there exist a unique language generated by that regular expression.

Example

Consider the language, defined over

Σ = {a , b} of words having at least one a, may be expressed by a regular expression (a+b)*a(a+b)*.

Consider the language, defined over Σ = {a, b} of words having at least one a and one b, may be expressed by a regular expression (a+b)*a(a+b)*b(a+b)*+ (a+b)*b(a+b)*a(a+b)*.

Consider the language, defined over Σ ={a, b}, of words starting with double a and ending in double b then its regular expression may be aa(a+b)*bb

Consider the language, defined over Σ ={a, b} of words starting with a and ending in b OR starting with b and ending in a, then its regular expression may be a(a+b)*b+b(a+b)*a

Theory of Automata

Lecture N0. 4

Reading Material

Introduction to Computer Theory Chapter 4, 5

Summary

Regular expression of EVEN-EVEN language, Difference between a* + b* and (a+b)*, Equivalent regular expressions; sum, product and closure of regular expressions; regular

languages, finite languages are regular, introduction to finite automaton, definition of FA, transition table, transition diagram

An important example

The Language EVEN-EVEN

Language of strings, defined over Σ={a, b} having even number of a's and even number of b's. i.e. EVEN-EVEN = {Λ, aa, bb, aaaa,aabb,abab, abba, baab, baba, bbaa, bbbb,…}, its regular expression can be written as (aa+bb+(ab+ba)(aa+bb)*(ab+ba))*

Note

It is important to be clear about the difference of the following
regular expressions $r_1 = a^*+b^*$ $r_2 = (a+b)^*$

Here r_1 does not generate any string of concatenation of a and b, while r_2 generates such strings.

Equivalent Regular Expressions

Definition

Two regular expressions are said to be equivalent if they generate the same language.

Example

Consider the following regular expressions $r_1 = (a + b)^*$ (aa + bb) $r_2 = (a + b)^*aa + (a + b)^*bb$ then both regular expressions define the language of strings ending in aa or bb.

Note

If $r_1 = (aa + bb)$ and $r_2 = (a + b)$
then $r_1+r_2 = (aa + bb) + (a + b)$
$r_1r_2 = (aa + bb)\ (a + b) = (aaa +$
aab + bba + bbb)

$(r_1)^* = (aa + bb)^*$

Regular Languages

Definition

The language generated by any regular expression is called a **regular language**.

It is to be noted that if r_1, r_2 are regular expressions, corresponding to the languages L_1 and L_2 then the languages generated by $r_1+ r_2$, r_1r_2(or r_2r_1) and r_1^*(or r_2^*) are also regular languages.

Note

It is to be noted that if L_1 and L_2 are expressed by $r_1$and r_2, respectively then the language expressed by $r_1+ r_2$, is the language $L_1 + L_2$ or $L_1 \cup L_2$

r_1r_2, , is the language L_1L_2, of strings obtained by prefixing every string of L_1 with every string of L_2 r_1*, is the language L_1*, of strings obtained by concatenating the strings of L, including the null string.

Example

If r_1 = (aa+bb) and r_2 = (a+b) then the language of strings generated by r_1+r_2, is also a regular language, expressed by (aa+bb) + (a+b)

If r_1 = (aa+bb) and r_2 = (a+b) then the language of strings generated by r_1r_2, is also a regular language, expressed by (aa+bb)(a+b)

If r = (aa+bb) then the language of strings generated by r*, is also a regular language, expressed by (aa+bb)*

All finite languages are regular

Example

Consider the language L, defined over Σ = {a,b}, of strings of length 2, starting with a, then

L = {aa, ab}, may be expressed by the regular expression aa+ab. Hence L, by definition, is a regular language.

Note

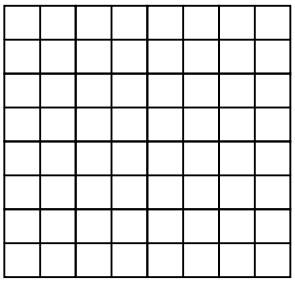
It may be noted that if a language contains even thousand words, its RE may be expressed, placing ' + ' between all the words.

Here the special structure of RE is not important.

Consider the language L = {aaa, aab, aba, abb, baa, bab, bba, bbb}, that may be expressed by a RE aaa+aab+aba+abb+baa+bab+bba+bbb, which is equivalent to (a+b)(a+b)(a+b).

Introduction to Finite Automaton

Consider the following game board that contains 64 boxes



There are some pieces of paper. Some are of white colour while others are of black colour. The number of pieces of paper are 64 or less. The possible arrangements under which these pieces of paper can be placed in the boxes, are finite. To start the game, one of the arrangements is supposed to be initial arrangement. There is a pair of dice that can generate the numbers 2,3,4,…12 . For each number generated, a unique arrangement is associated among the possible arrangements.

It shows that the total number of transition rules of arrangement are finite. One and more arrangements can be supposed to be the winning arrangement. It can be observed that the winning of the game depends on the sequence in which the numbers are generated. This structure of game can be considered to be a finite automaton.

Method 4 (Finite Automaton)

Definition

A Finite automaton (FA), is a collection of the followings

Finite number of states, having one initial and some (maybe none) final states.

Finite set of input letters (Σ) from which input strings are formed.

Finite set of transitions i.e. for each state and for each input letter there is a transition showing how to move from one state to another.

MAS ALL ROUNDER

Example

Σ = {a,b}

States: x, y, z where x is an initial state and z is final state.

Transitions:

At state x reading a, go to state z

At state x reading b, go to state y

At state y reading a, b go to state y

At state z reading a, b go to state z

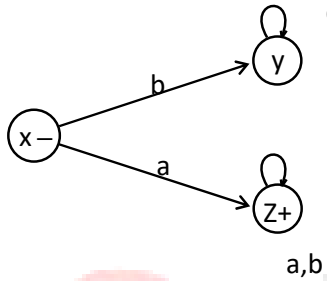
These transitions can be expressed by the following table called transition table

Old States	New States

	Reading a	Reading b
x -	z	y
y	y	y
z +	z	z

Note

It may be noted that the information of an FA, given in the previous table, can also be depicted by the following diagram, called the **transition diagram**, of the given FA a,b



Remark

The above transition diagram is an FA accepting the language of strings, defined over Σ = {a, b}, **starting with a**. It may be noted that this language may be expressed by the regular expression a(a + b)*

MAS ALL ROUNDER

Theory of Automata

Lecture N0. 5

Reading Material

Introduction to Computer Theory Chapter 5

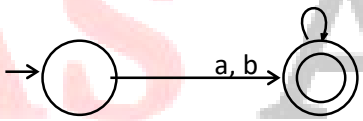
Summary

Different notations of transition diagrams, languages of strings of **even length, Odd length, starting with b, ending in a, beginning with b, not beginning with b, beginning and ending in same letters**

Note

It may be noted that to indicate the initial state, an arrow head can also be placed before that state and that the

final state with double circle, as shown below. It is also to be noted that while expressing an FA by its transition diagram, the labels of states are not necessary. a, b



Example Σ = {a,b}

States: x, y, where x is both initial and final state.

Transitions:

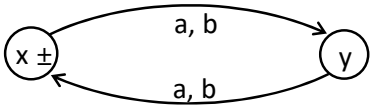
At state x reading a or b go to state y.

At state y reading a or b go to state x.

These transitions can be expressed by the following transition table

	New States	
Old States	Reading a	Reading b
x ±	y	y
y	x	x

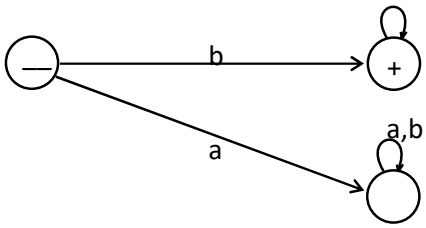
It may be noted that the above transition table may be depicted by the following transition diagram.



The above transition diagram is an FA accepting the language of strings, defined over Σ={a, b} of **even length**. It may be noted that this language may be expressed by the regular expression $((a+ b) (a + b))^*$

Example:

Consider the language L of strings, defined over Σ={a, b}, **starting with b.** The language L may be expressed by RE b(a + b)*, may be accepted by the following FA a,b



MAS ALL

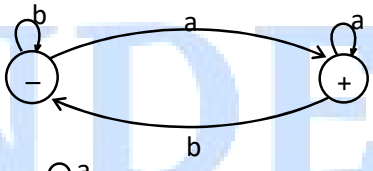
1

Example

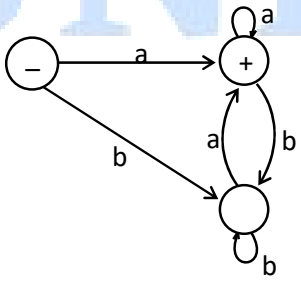
Consider the language L of strings, defined over Σ={a, b}, **ending in a. T**he language L may be expressed by RE $(a+b)^*a$.

This language may be accepted by the

FA shown aside



ROUNDER



There may be another FA corresponding to the given language, as shown aside

Note

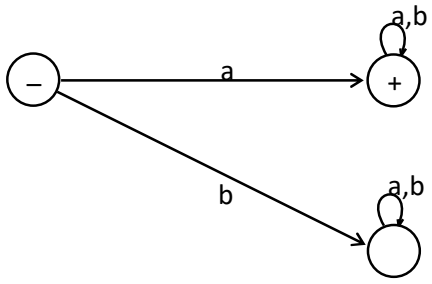
It may be noted that corresponding to a given language there may be more than one FA accepting that language, but for a given FA there is a unique language accepted by that FA. It is also to be noted that given the languages L_1 and L_2 ,where

L_1 = The language of strings, defined over Σ ={a, b}, **beginning with a.**

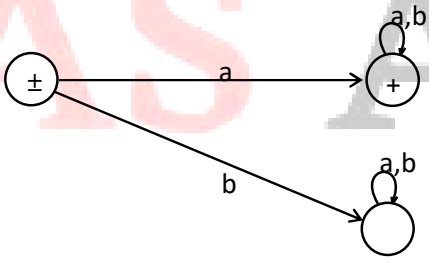
L_2 = The language of strings, defined over Σ ={a, b}, **not beginning with b**

The Λ does not belong to L_1 while it does belong to L_2. This fact may be depicted by the corresponding transition diagrams of L_1 and L_2.

FA_1 Corresponding to L



The language Lmay be expressed by the regular expression $a(a + b)^*$



FA_2 Corresponding to L_2

The language L_2 may be expressed by the regular expression $a(a + b)^* + Λ$

Example

Consider the Language L of Strings of **length two or more**, defined over Σ = {a, b}, **beginning with and ending in same letters.**

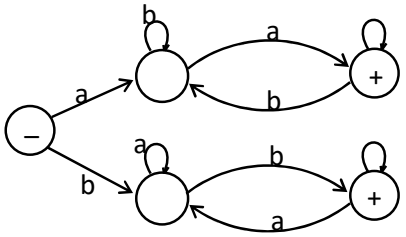
The language L may be expressed by the following regular expression $a(a + b)^*a + b(a + b)^*b$

It is to be noted that if the condition on the length of string is not imposed in the above language then **the**

strings a and b will then belong to the language.

a

This language L may be accepted by the FA as shown aside



b

Theory of Automata

Lecture N0. 6

Reading Material

Introduction to Computer Theory Chapter 5

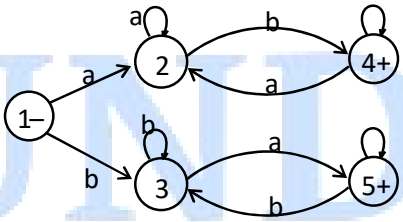
Summary

Language of strings beginning with and ending in different letters, Accepting all strings, accepting nonempty strings, accepting no string, containing double a's, having double 0's or double 1's, containing triple a's or triple b's, EVEN-EVEN

Example

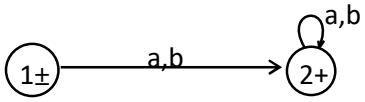
Consider the Language L of Strings, defined over Σ = {a, b}, **beginning with and ending in different letters.** The language L may be expressed by the following regular expression a(a + b)*b + b(a + b)*a

This language may be accepted by the following FA



b

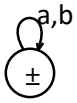
a



Example

Consider the Language L, defined over Σ = {a, b}

of **all strings including Λ**. The language L may be accepted by the following FA



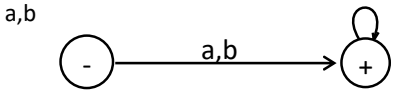
The language L may also be accepted by the following FA

The language L may be expressed by the regular expression $(a + b)^*$

Example

Consider the Language L , defined over Σ = {a, b} of **all non empty strings**. The language L may be accepted

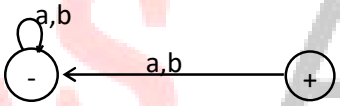
by the following FA



The above language may be expressed by the regular expression $(a + b)^+$

Example

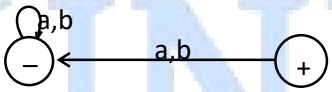
Consider the following FA, defined over Σ = {a, b}



It is to be noted that the above FA **does not accept any string,** even it does not accept the null string; as there is no path starting from initial state and ending in final state.

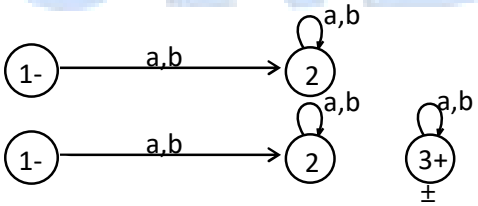
Equivalent FAs

It is to be noted that two FAs are said to be equivalent, if they accept the same language, as shown in the following FAs.



FA_1

FA_2



FA_3

Note

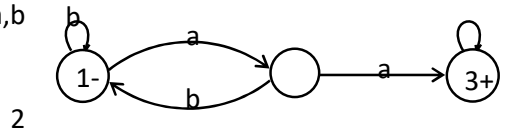
FA_1 has already been discussed, while in FA_2, there is no final state and in FA_3, there is a final state but FA_3 is disconnected as the states 2 and 3 are disconnected.

It may also be noted that the language of strings accepted by FA_1, FA_2 and FA_3 is denoted by the empty set *i.e.* { } OR Ø

Example

Consider the Language L of strings , defined over Σ = {a, b}, **containing double a.**

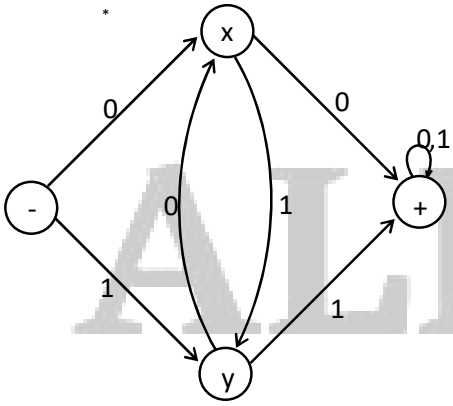
The language L may be expressed by the regular expression $(a+b)^*$ (aa) $(a+b)^*$. This language may be accepted by the following FA. a,b

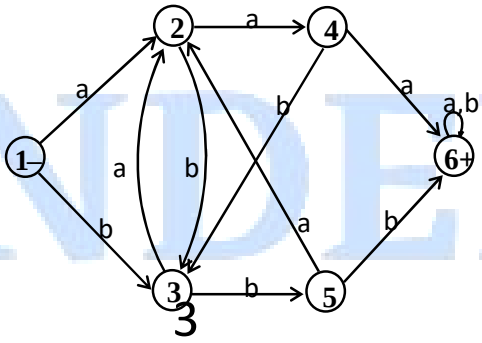


Example

Consider the language L of strings, defined over Σ={0, 1}, **having double 0's or double 1's,** The language L may be expressed by the regular expression $(0+1)^*$ (00 + 11) (0+1)

This language may be accepted by the following FA

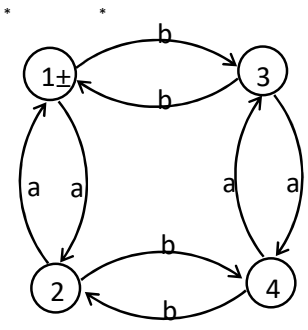




Example

Consider the language L of strings, defined over Σ={a, b}, **having triple a's or triple b's.** The language L may be expressed by RE $(a+b)^*$ (aaa + bbb) $(a+b)^*$

This language may be accepted by the FA as shown aside



Example

Consider the **EVEN-EVEN** language, defined over Σ = {a, b}. As discussed earlier that **EVEN-EVEN** language can be expressed by the regular expression (aa+bb+(ab+ba)(aa+bb) (ab+ba))

EVEN-EVEN language may be accepted by the FA as shown aside

MAS ALL ROUNDER

Theory of Automata

Introduction to Computer Theory

Lecture N0. 7

Reading Material

Chapter 5, 6

Summary

FA corresponding to finite languages(using both methods), Transition graphs.

FA corresponding to finite languages

Example

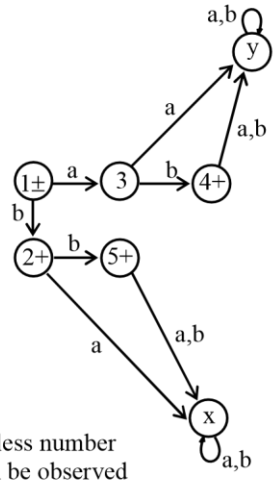
Consider the language

© Copyright Virtual University of Pakistan

Theory of Automata

Introduction to Computer Theory

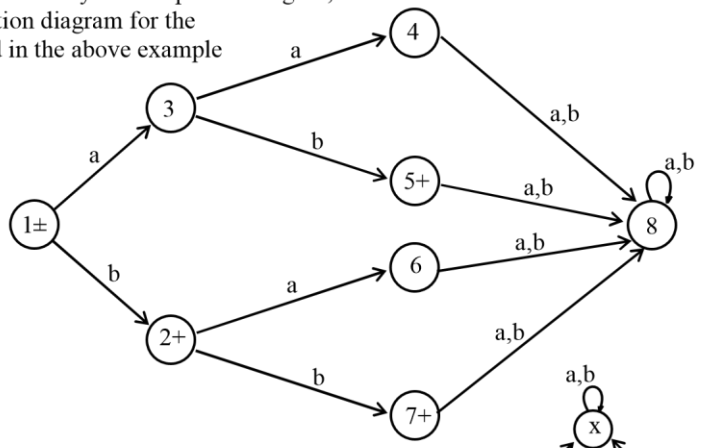
L = {Λ, b, ab, bb}, defined over Σ ={a, b}, expressed by Λ + b + ab + bb OR Λ + b (Λ + a + b). The language L may be accepted by the FA as shown aside



It is to be noted that the states x and y are called **Dead States, Waste Baskets or Davey John Lockers,** as the moment one enters these states there is no way to leave it.

Note

It is to be noted that to build an FA accepting the language having less number of strings, the tree structure may also help in this regard, which can be observed in the following transition diagram for the Language L, discussed in the above example



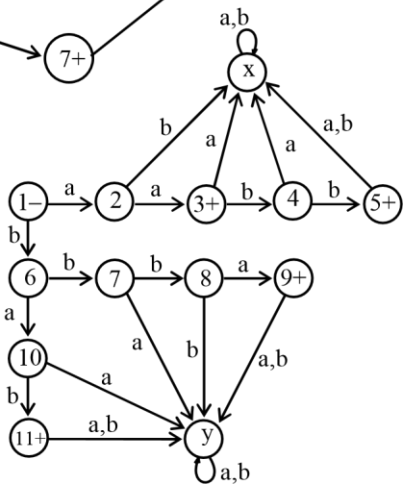
Example

Consider the language

L = {aa, bab, aabb, bbba}, defined over Σ ={a, b}, expressed by aa + bab + aabb + bbba

OR aa (Λ + bb) + b (ab + bba)

The above language may be accepted by the FA as shown aside



© Copyright Virtual University of Pakistan

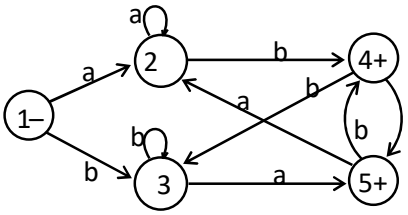
Theory of Automata

Introduction to Computer Theory

Consider the language L = {w belongs to {a,b}*: length(w) ≥2 and w neither ends in **aa** nor **bb**}.

The language L may be expressed by the regular expression (a+b)*(ab+ba)

This language may be accepted by the following FA



a

© Copyright Virtual University of Pakistan

Theory of Automata

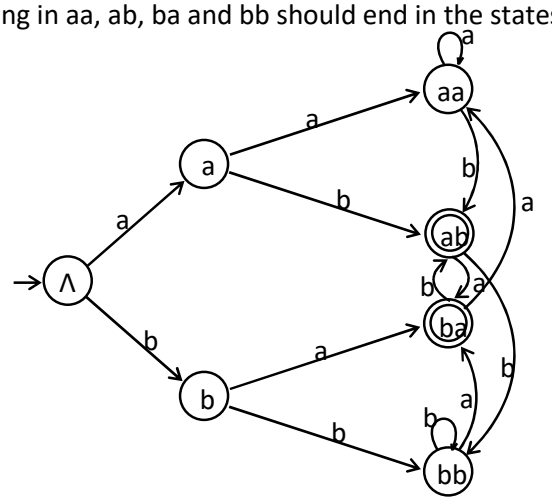
Introduction to Computer Theory

Note

It is to be noted that building an FA corresponding to the language L, discussed in the above example, seems to be quite difficult, but the same can be done using tree structure along with the technique discussed in the book

Introduction to Languages and Theory of Computation, by J. C. Martin

so that the strings ending in aa, ab, ba and bb should end in the states labeled as aa, ab, ba and bb, respectively;



as shown in the following FA

© Copyright Virtual University of Pakistan

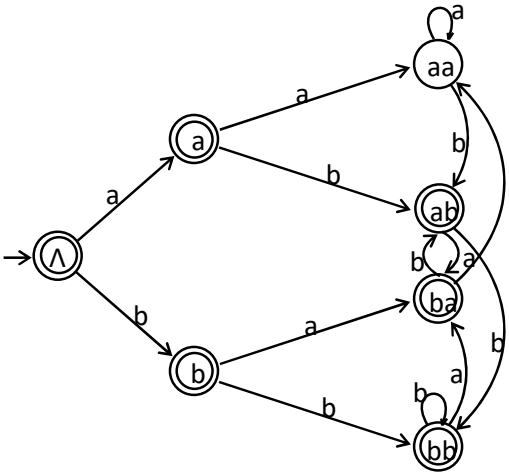
Theory of Automata

Introduction to Computer Theory

Example

Consider the language FA corresponding to r_1+r_2 can be determined as L = {w belongs to {a,b}*: w does not end in **aa**}.

The language L may be expressed by the regular expression Λ + a + b + (a+b)*(ab+ba+bb). This language may be accepted by the following FA



a

Method 5 (Transition Graph)

Definition:

A Transition graph (TG), is a collection of the followings

Finite number of states, at least one of which is start state and some (maybe none) final states.

Finite set of input letters (Σ) from which input strings are formed.

Finite set of transitions that show how to go from one state to another based on reading specified substrings of input letters, possibly even the null string (Λ).

Lecture N0. 8

© Copyright Virtual University of Pakistan

Theory of Automata

Introduction to Computer Theory

Reading Material

Chapter 6

Summary

Examples of TGs: accepting all strings, accepting none, starting with b, not ending in b, containing aa, containing aa or bb

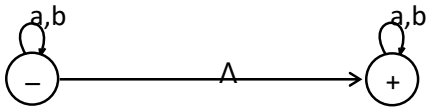
Note

It is to be noted that in TG there may exist more than one paths for certain string, while there may not exist any path for certain string as well. If there exists at least one path for a certain string, starting from initial state and ending in a final state, the string is supposed to be accepted by the TG, otherwise the string is supposed to be rejected. Obviously collection of accepted strings is the language accepted by the TG.

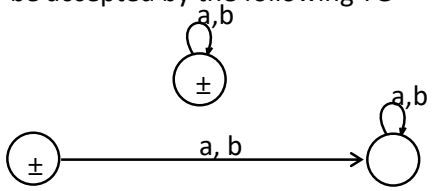
Example

Consider the Language L , defined over Σ = {a, b} of **all strings including Λ.** The language L may be accepted

by the following TG



The language L may also be accepted by the following TG



TG_1

TG_2 +

Example

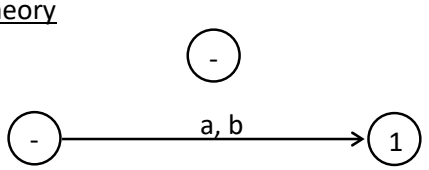
Consider the following TGs

© Copyright Virtual University of Pakistan

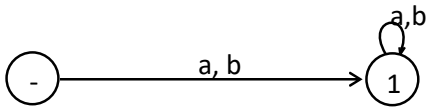
Theory of Automata

Introduction to Computer Theory

TG_1



TG_2



TG_3

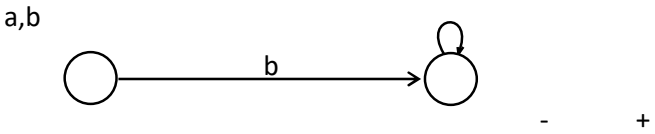
It may be observed that in the first TG, no transition has been shown. Hence this TG does not accept any string, defined over any alphabet. In TG_2 there are transitions for a and b at initial state but there is no transition at state 1. This TG still does not accept any string. In TG_3 there are transitions at both initial state and state 1, but it does not accept any string.

Thus none of TG_1, TG_2 and TG_3 accepts any string, *i.e.* these TGs accept empty language. It may be noted that TG_1 and TG_2 are TGs but not FA, while TG_3 is both TG and FA as well.

It may be noted that every FA is a TG as well, but the converse may not be true, *i.e.* every TG may not be an FA.

Example

Consider the language L of strings, defined over Σ={a, b}, **starting with b.** The language L may be expressed by RE $b(a + b)^*$, may be accepted by the following TG

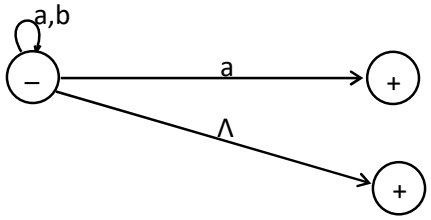


Consider the language L of strings, defined over Σ={a, b}, **not ending in b.** The language L may be expressed by RE $\Lambda + (a + b)^*a$, may be accepted by the following TG

© Copyright Virtual University of Pakistan

Theory of Automata

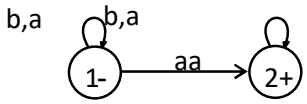
Introduction to Computer Theory



Example

Consider the Language L of strings, defined over Σ = {a, b}, **containing double a.**

The language L may be expressed by the following regular expression $(a+b)^*(aa)(a+b)^*$. This language may be accepted by the following TG

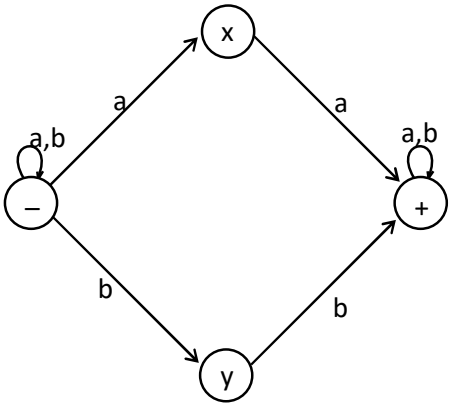


Example

Consider the language L of strings, defined over Σ={a, b}, **having double a or double b**.

The language L can be expressed by RE $(a+b)^*(aa + bb)(a+b)^*$.

The above language may also be expressed by the following TGs.

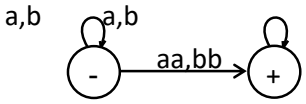


OR

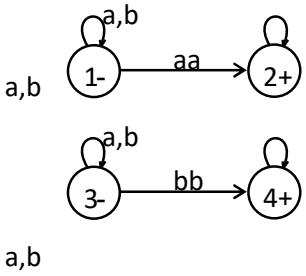
© Copyright Virtual University of Pakistan

Theory of Automata

Introduction to Computer Theory



OR



Note

In the above TG if the states are not labeled then it may not be considered to be a single TG

Lecture N0. 9
Reading Material

Chapter 6

Summary

TGs accepting the languages: containing aaa or bbb, beginning and ending in different letters, beginning and ending in same letters, EVEN-EVEN, a's occur in even clumps and ends in three or more b's, example showing different paths traced by one string, Definition of GTG

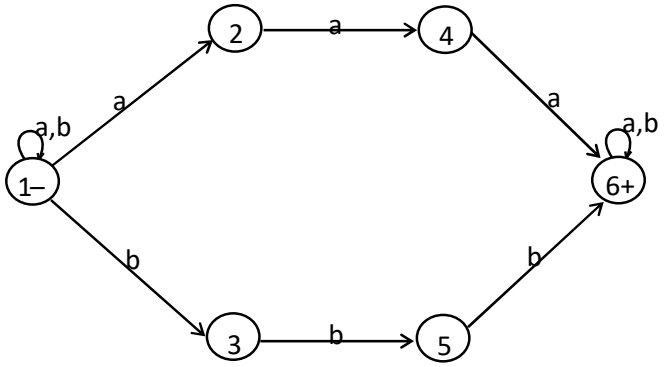
Example

Consider the language L of strings, defined over Σ = {a, b}, **having triple a or triple b.** The language L may be expressed by RE $(a+b)^*$ (aaa + bbb) $(a+b)^*$

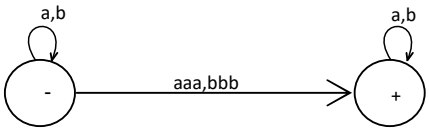
© Copyright Virtual University of Pakistan

Introduction to Computer Theory

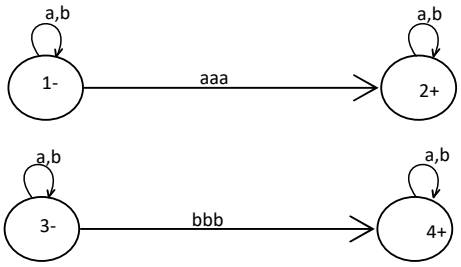
This language may be accepted by the following TG



OR



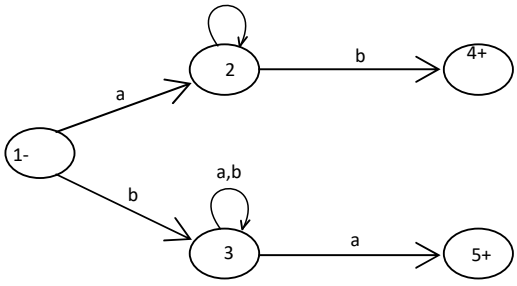
OR



Example

Consider the language L of strings, defined over Σ = {a, b}, **beginning and ending in different letters**.

The language L may be expressed by RE $a(a + b)^*b + b(a + b)^*a$ a, b The language L may be accepted by the following TG



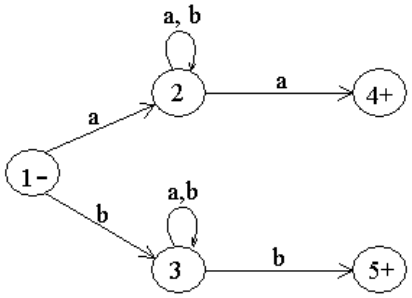
© Copyright Virtual University of Pakistan

Theory of Automata

Introduction to Computer Theory

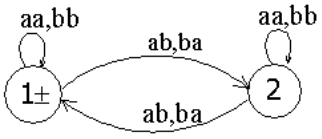
Consider the Language L of strings **of length two or more**, defined over Σ = {a, b}, **beginning with and ending in same letters.**

The language L may be expressed by the following regular expression $a(a + b)^*a + b)^*b$ This language may be accepted by the following TG



Example

Consider the **EVEN-EVEN** language, defined over Σ = {a, b}. As discussed earlier that **EVEN-EVEN** language can be expressed by a regular expression $(aa+bb+(ab+ba)(aa+bb)^*(ab+ba))^*$ The language **EVEN-EVEN** may be accepted by the following TG



Example

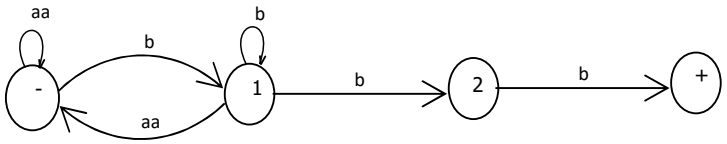
Consider the language L, defined over Σ={a, b}, in which **a's occur only in even clumps and that ends in three or more b's**. The language L can be expressed by its regular expression $(aa)^*b(b^*+(aa(aa)^*b)^*)$ bb OR $(aa)^*b(b^*+((aa)^+b)^*)$ bb.

The language L may be accepted by the following TG

© Copyright Virtual University of Pakistan

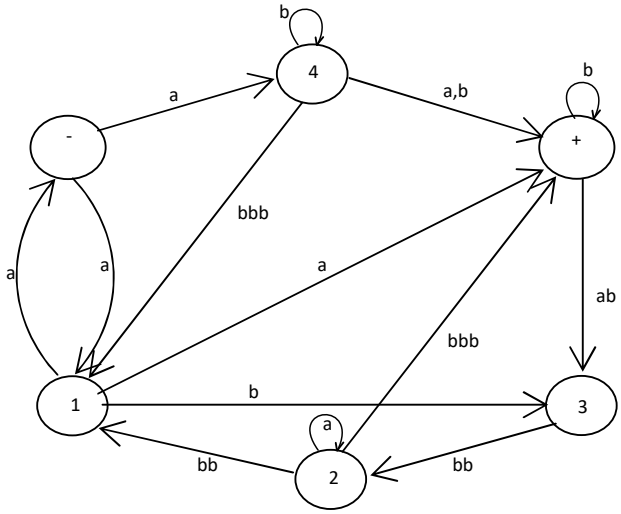
Theory of Automata

Introduction to Computer Theory



Example

Consider the following TG



© Copyright Virtual University of Pakistan

Consider the string abbbabbbabba. It may be observed that the above string traces the following three paths,

(using the states)

(a)(b) (b) (b) (ab) (bb) (a) (bb) (a)

(-)(4)(4)(+)(+)(3)(2)(2)(1)(+)

(a)(b) ((b)(b)) (ab) (bb) (a) (bb) (a)

(-)(4)(+)(+)(+)(3)(2)(2)(1)(+)

(a)((b) (b)) (b) (ab) (bb) (a) (bb) (a)

(-) (4)(4)(4)(+) (3)(2)(2)(1)(+)

Which shows that all these paths are successful, (*i.e.* the path starting from an initial state and ending in a final state).

Hence the string abbbabbbabba is accepted by the given TG.

MAS ALL

Generalized Transition Graphs

A generalized transition graph (GTG) is a collection of three things

Finite number of states, at least one of which is start state and some (maybe none) final states.

Finite set of input letters (Σ) from which input strings are formed.

Directed edges connecting some pair of states labeled with regular expression.

It may be noted that in GTG, the labels of transition edges are corresponding regular expressions

ROUNDER

Theory of Automata

Lecture N0. 10

Reading Material

Introduction to Computer Theory Chapter 6, 7

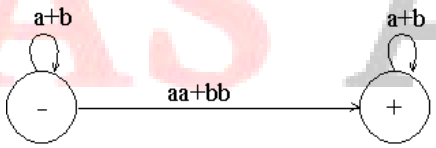
Summary

Examples of GTG accepting the languages of strings: containing aa or bb, beginning with and ending in same letters, beginning with and ending in different letters, containing aaa or bbb,

Nondeterminism, Kleene's theorem (part I, part II, part III), proof of Kleene's theorem part I

Example

Consider the language L of strings, defined over Σ = {a,b}, containing **double a or double b**. The language L can be expressed by the following regular expression $(a+b)^*$ (aa + bb) $(a+b)^*$ The language L may be accepted by the following GTG.

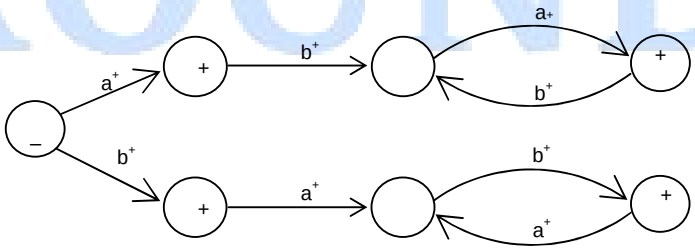


Example

Consider the Language L of strings, defined over Σ = {a, b}, **beginning with and ending in same letters.**

The language L may be expressed by the following regular expression (a+b)+ $a(a + b)^*a + b(a + b)^*b$. This language may be accepted by the following GTG

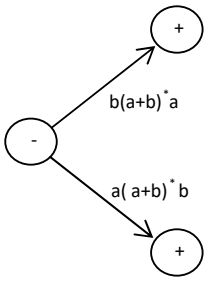
Example



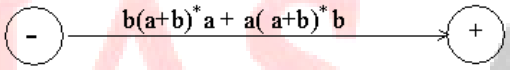
Example

Consider the language L of strings of, defined over Σ = {a, b}, **beginning and ending in different letters**.

The language L may be expressed by RE $a(a + b)^*b + b(a + b)^*a$ The language L may be accepted by the following GTG



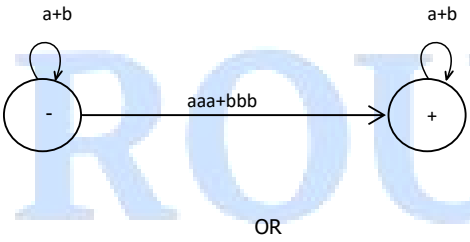
The language L may be accepted by the following GTG as well



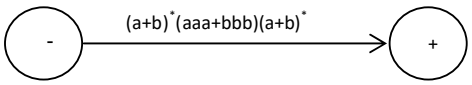
MAS ALL ROUNDER

Example

Consider the language L of strings, defined over Σ = {a, b}, **having triple a or triple b.** The language L may be expressed by RE $(a+b)^*(aaa + bbb)(a+b)^*$ This language may be accepted by the following GTG



OR



Nondeterminism

TGs and GTGs provide certain relaxations *i.e.* there may exist more than one path for a certain string or there may not be any path for a certain string, this property creates **nondeterminism** and it can also help in differentiating TGs or GTGs from FAs. Hence an FA is also called a Deterministic Finite Automaton (DFA).

Kleene's Theorem

If a language can be expressed by

FA or

TG or RE

then it can also be expressed by other two as well.

It may be noted that the theorem is proved, proving the following three parts

Kleene's Theorem Part I

If a language can be accepted by an FA then it can be accepted by a TG as well.

Kleene's Theorem Part II

If a language can be accepted by a TG then it can be expressed by an RE as well.

Kleene's Theorem Part III

If a language can be expressed by a RE then it can be accepted by an FA as well.

Proof(Kleene's Theorem Part I)

Since every FA can be considered to be a TG as well, therefore there is nothing to prove.

Theory of Automata

Lecture N0. 11

Reading Material

Introduction to Computer Theory Chapter 7

Summary

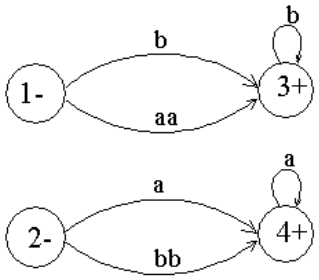
proof of Kleene's theorem part II (method with different steps), particular examples of TGs to determine corresponding REs.

Proof(Kleene's Theorem Part II)

To prove part II of the theorem, an algorithm consisting of different steps, is explained showing how a RE can be obtained corresponding to the given TG. For this purpose the notion of TG is changed to that of GTG *i.e.* the labels of transitions are corresponding REs.

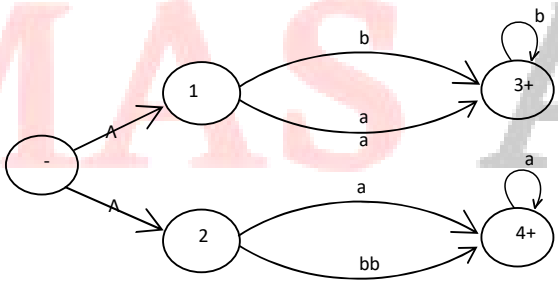
Basically this algorithm converts the given TG to GTG with one initial state along with a single loop, or one initial state connected with one final state by a single transition edge. The label of the loop or the transition edge will be the required RE.

Step 1 If a TG has more than one start states, then introduce a new start state connecting the new state to the old start states by the transitions labeled by Λ and make the old start states the non-start states. This step can be shown by the following example



Example

The above TG can be converted to

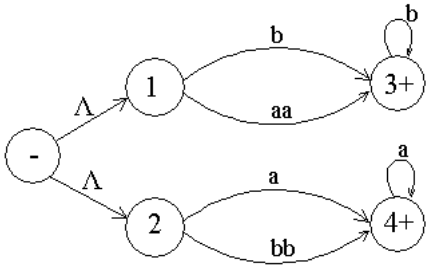


Step 2:

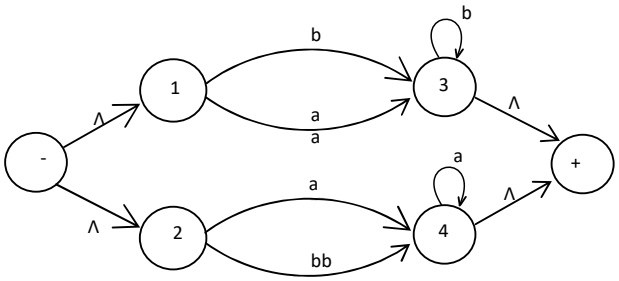
If a TG has more than one final states, then introduce a new final state, connecting the old final states to the new final state by the transitions labeled by Λ.

This step can be shown by the previous example of TG, where the step 1 has already been processed

Example



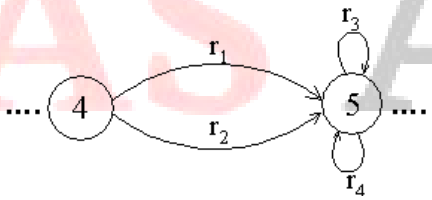
The above TG can be converted to



Step 3:

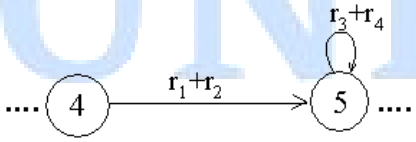
If a state has two (more than one) incoming transition edges labeled by the corresponding REs, from the same state (including the possibility of loops at a state), then replace all these transition edges with a single transition edge labeled by the sum of corresponding REs.

This step can be shown by a part of TG in the following example



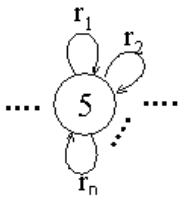
Example

The above TG can be reduced to

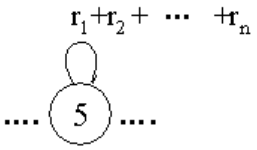


Note

The step 3 can be generalized to any finite number of transitions as shown below



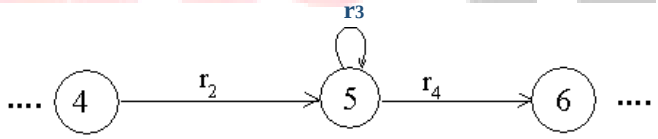
The above TG can be reduced to



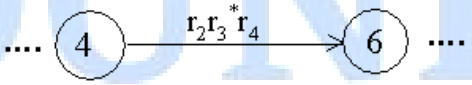
Step 4 (bypass and state elimination)

If three states in a TG, are connected in sequence then eliminate the middle state and connect the first state with the third by a single transition (include the possibility of circuit as well) labeled by the RE which is the concatenation of corresponding two REs in the existing sequence. This step can be shown by a part of TG in the following example

Example



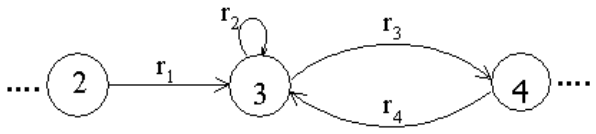
To eliminate state 5 the above can be reduced to



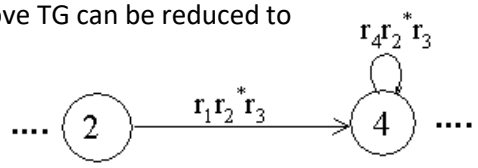
Consider the following example containing a circuit

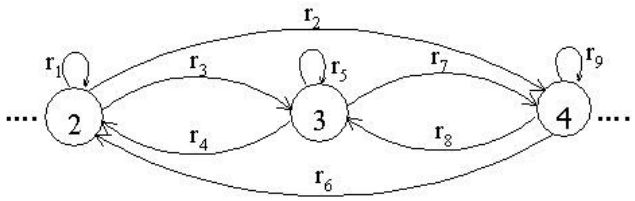
Example

Consider the part of a TG, containing a circuit at a state, as shown below



To eliminate state 3 the above TG can be reduced to



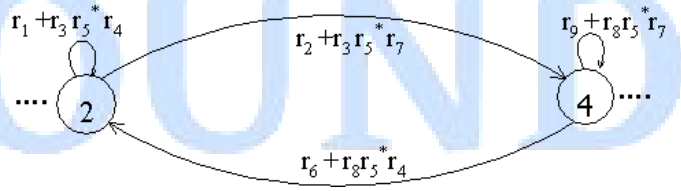


Example

Consider a part of the following TG

MAS ALL

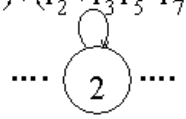
To eliminate state 3 the above TG can be reduced to



ROUNDER

To eliminate state 4 the above TG can be reduced to

$$(r_1 + r_3 r_5^* r_4) + (r_2 + r_3 r_5^* r_7)(r_9 + r_8 r_5^* r_7)^*(r_6 + r_8 r_5^* r_4)$$

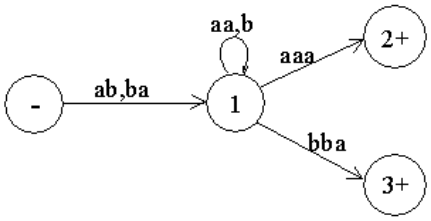


Note

It is to be noted that to determine the RE corresponding to a certain TG, four steps have been discussed. This process can be explained by the following particular examples of TGs

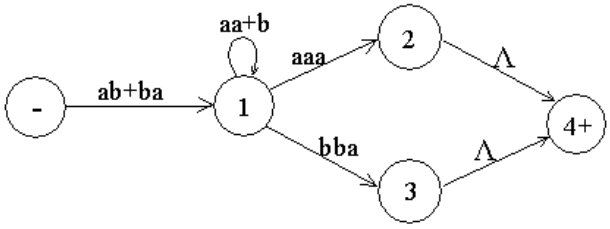
Example

Consider the following

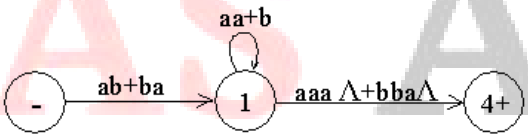


TG

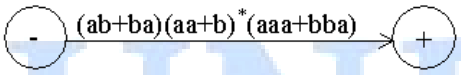
To have single final state, the above TG can be reduced to the following



To eliminate states 2 and 3, the above TG can be reduced to the following



To eliminate state 1 the above TG can be reduced to the following



Hence the required RE is $(ab+ba)(aa+b)^*(aaa+bba)$

MAS ALL ROUNDER

Theory of Automata

Lecture N0. 12

Reading Material

Introduction to Computer Theory Chapter 7

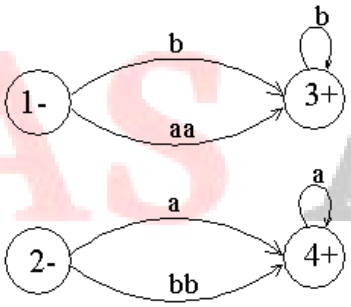
Summary

Examples of writing REs to the corresponding TGs, RE corresponding to TG accepting EVEN-EVEN language,

Kleene's theorem part III (method 1:union of FAs), examples of FAs corresponding to simple REs, example of Kleene's theorem part III (method 1) continued

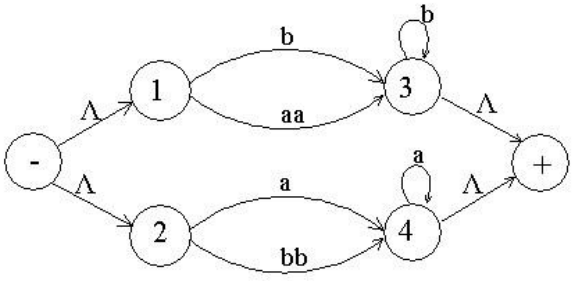
Example

Consider the following TG

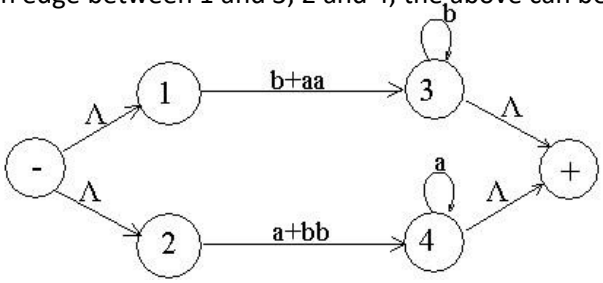


MAS ALL ROUNDER

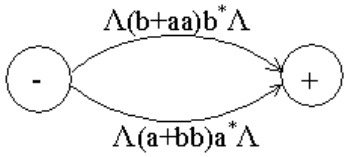
To have single initial and single final state the above TG can be reduced to the following



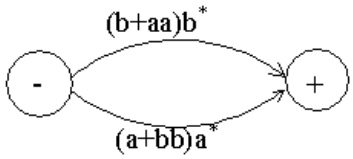
To obtain single transition edge between 1 and 3; 2 and 4, the above can be reduced to the following



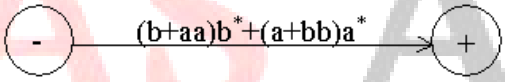
To eliminate states 1,2,3 and 4, the above TG can be reduced to the following TG



OR



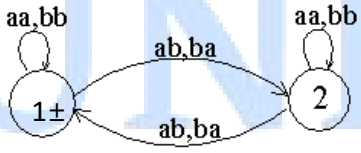
To connect the initial state with the final state by single transition edge, the above TG can be reduced to the following



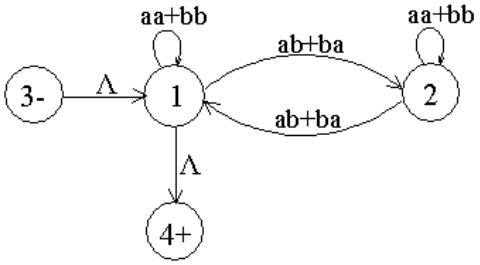
Hence the required RE is $(b+aa)b^*+(a+bb)a^*$

Example

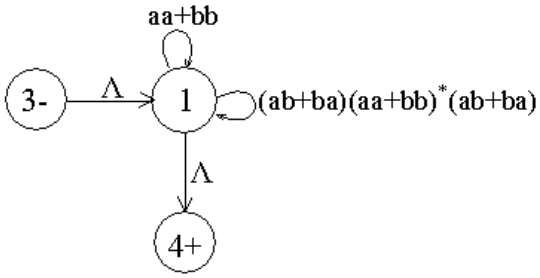
Consider the following TG, accepting EVEN-EVEN language



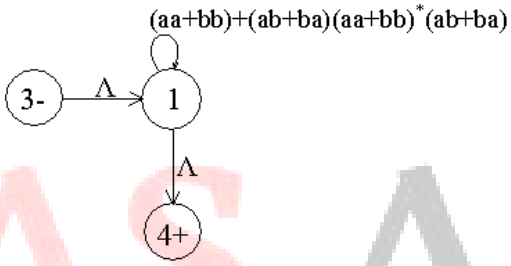
It is to be noted that since the initial state of this TG is final as well and there is no other final state, so to obtain a TG with single initial and single final state, an additional initial and a final state are introduced as shown in the following TG



To eliminate state 2, the above TG may be reduced to the following

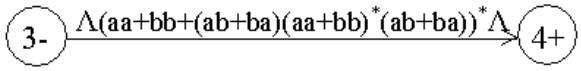


To have single loop at state 1, the above TG may be reduced to the following



MAS ALL

To eliminate state 1, the above TG may be reduced to the following



ROUNDER

Hence the required RE is $(aa+bb+(ab+ba)(aa+bb)^*(ab+ba))^*$

Kleene's Theorem Part III

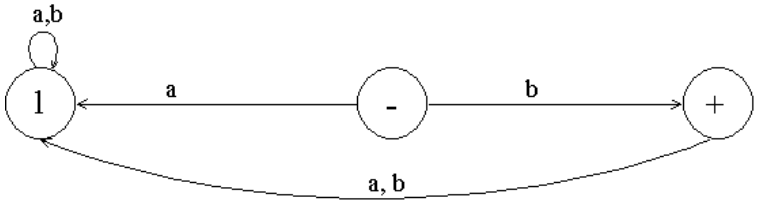
Statement:

If the language can be expressed by a RE then there exists an FA accepting the language.

As the regular expression is obtained applying addition, concatenation and closure on the letters of an alphabet and the Null string, so while building the RE, sometimes, the corresponding FA may be built easily, as shown in the following examples

Example

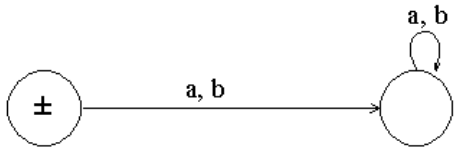
Consider the language, defined over Σ = {a,b}, **consisting of only b**, then this language may be accepted by the following FA



which shows that this FA helps in building an FA accepting only one letter

Example

Consider the language, **consisting of only** ∧, then accepted by the following FA



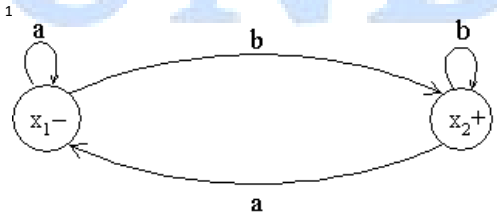
defined over Σ = {a,b}, this language may be

As, if r_1 and r_2 are regular expressions then their sum, concatenation and closure are also regular expressions, so an FA can be built for any regular expression if the methods can be developed for building the FAs corresponding to the sum, concatenation and closure of the regular expressions along with their FAs. These three methods are explained in the following discussion

Method1 (Union of two FAs): Using the FAs corresponding to r_1 and r_2 an FA can be built, corresponding to $r_1 + r_2$. This method can be developed considering the following examples

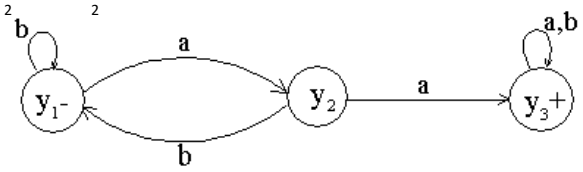
Example

Let $r_1 = (a+b)^*b$ defines L_1



and the FA be

and $r_2 = (a+b)^*aa(a+b)^*$

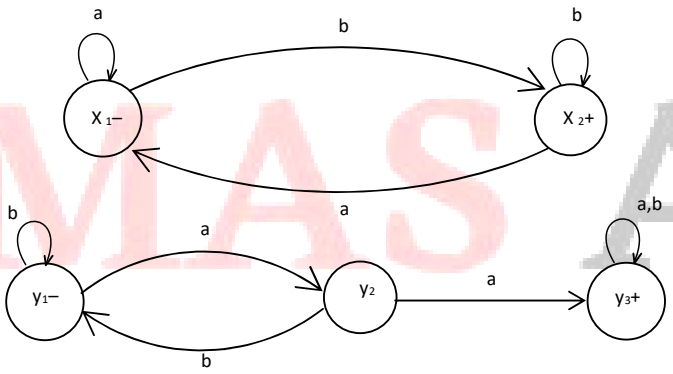


defines L and FA be

Let FA_3 be an FA corresponding to $r_1 + r_2$, then the initial state of FA_3 must correspond to the initial state of FA_1 and the initial state of FA_2.

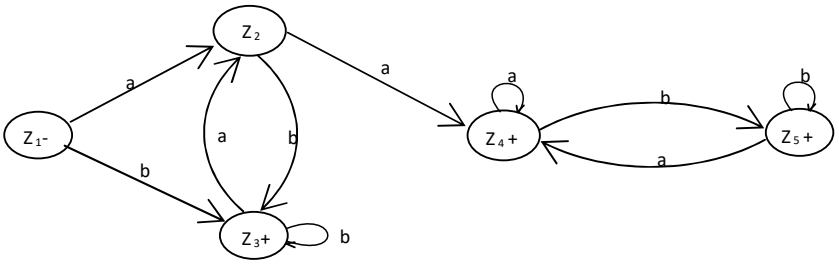
Since the language corresponding to $r_1 + r_2$ is the union of corresponding languages L_1 and L_2, consists of the strings belonging to L_1 or L_2 or both, therefore a final state of FA_3 must correspond to a final state of FA_1 or FA_2 or both.

Since, in general, FA_3 will be different from both FA_1 and FA_2, so the labels of the states of FA_3 may be supposed to be $z_1, z_2, z_3, \ldots$, where z_1 is supposed to be the initial state. Since z_1 corresponds to the states x_1 or y_1, so there will be two transitions separately for each letter read at z_1. It will give two possibilities of states either z_1 or different from z_1. This process may be expressed in the following transition table for all possible states of
FA_3.



MAS ALL ROUNDER

Old States	New States after reading	
	a	b
$z_1 - \equiv (x_1, y_1)$	$(x_1, y_2) \equiv z_2$	$(x_2, y_1) \equiv z_3$
$z_2 \equiv (x_1, y_2)$	$(x_1, y_3) \equiv z_4$	$(x_2, y_1) \equiv z_3$
$z_3 + \equiv (x_2, y_1)$	$(x_1, y_2) \equiv z_2$	$(x_2, y_1) \equiv z_3$
$z_4 + \equiv (x_1, y_3)$	$(x_1, y_3) \equiv z_4$	$(x_2, y_3) \equiv z_5$
$z_5 + \equiv (x_2, y_3)$	$(x_1, y_3) \equiv z_4$	$(x_2, y_3) \equiv z_5$



RE corresponding to the above FA may be $r_1+r_2 = (a+b)^*b + (a+b)^*aa(a+b)^*$.
Note: Further examples are discussed in the next lecture.

Theory of Automata

Lecture N0. 13

Reading Material

Introduction to Computer Theory Chapter 7

Summary

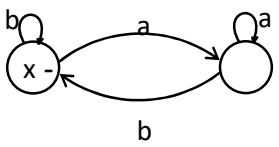
Examples of Kleene's theorem part III (method 1) continued, Kleene's theorem part III (method 2: Concatenation of FAs), Example of Kleene's theorem part III (method 2 : Concatenation of FAs)

Note

It may be noted that the example discussed at the end of previous lecture, FA_1 contains two states while FA_2 contains three states. Hence the total number of possible combinations of states of FA_1 and FA_2, in sequence, will be six. For each combination the transitions for both a and b can be determined, but using the method in the example, number of states of FA_3 was reduced to five.

Example

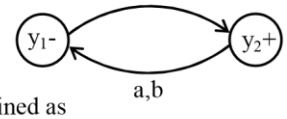
Let $r_1 = (a+b)^*a$ and the corresponding FA_1 be



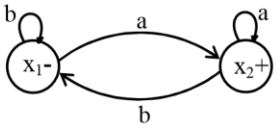
1 x_2 +

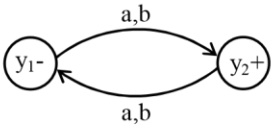
also $r_2 = (a+b)((a+b)(a+b))^*$ or $((a+b)(a+b))^*(a+b)$ and FA_2 be

a,b

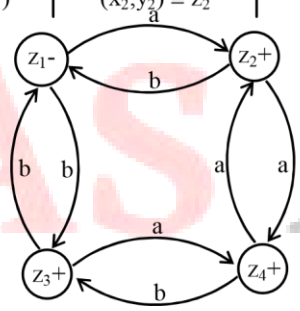


FA corresponding to r_1+r_2 can be determined as





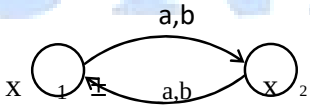
Old States	New States after reading	
	a	b
$z_1 \equiv (x_1,y_1)$	$(x_2,y_2) \equiv z_2$	$(x_1,y_2) \equiv z_3$
$z_2 + \equiv (x_2,y_2)$	$(x_2,y_1) \equiv z_4$	$(x_1,y_1) \equiv z_1$
$z_3 + \equiv (x_1,y_2)$	$(x_2,y_1) \equiv z_4$	$(x_1,y_1) \equiv z_1$
$z_4 + \equiv (x_2,y_1)$	$(x_2,y_2) \equiv z_2$	$(x_1,y_2) \equiv z_3$



MAS ALL ROUNDER

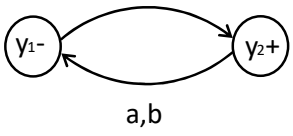
Example

Let $r_1 = ((a+b)(a+b))^*$ and the corresponding FA_1 be



also $r_2 = (a+b)((a+b)(a+b))^*$ or $((a+b)(a+b))^*(a+b)$ and FA_2 be

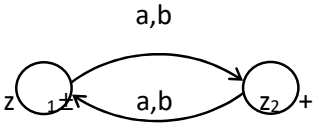
a,b



FA corresponding to r_1+r_2 can be determined as

Old States	New States after reading	
	a	b
$z_1\pm\equiv(x_1,y_1)$	$(x_2,y_2)\equiv z_2$	$(x_2,y_2)\equiv z_2$
$z_2+\equiv(x_2,y_2)$	$(x_1,y_1)\equiv z_1$	$(x_1,y_1)\equiv z_1$

Hence the required FA will be as follows



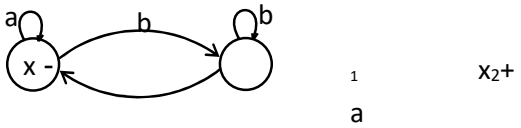
Method2 (Concatenation of two FAs):

Using the FAs corresponding to r_1 and r_2, an FA can be built, corresponding to r_1r_2. This method can be developed considering the following examples

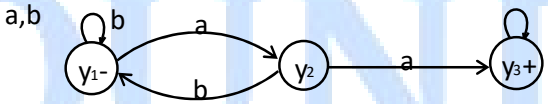
MAS ALL

Example

Let $r_1 = (a+b)^*b$ defines L_1 and FA_1 be



and $r_2 = (a+b)^*aa(a+b)^*$ defines L_2 and FA_2 be

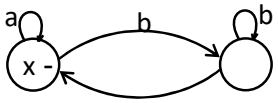


ROUNDER

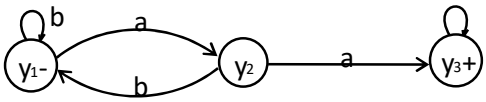
Let FA_3 be an FA corresponding to r_1r_2, then the initial state of FA_3 must correspond to the initial state of FA_1 and the final state of FA_3 must correspond to the final state of FA_2.Since the language corresponding to r_1r_2 is the concatenation of corresponding languages L_1 and L_2, consists of the strings obtained, concatenating the strings of L_1 to those of L_2, therefore ***the moment a final state of first FA is entered, the possibility of the initial state of second FA will be included as well.***

Since, in general, FA_3 will be different from both FA_1 and FA_2, so the labels of the states of FA_3 may be supposed to be $z_1, z_2, z_3, \ldots$, where z_1 stands for the initial state. Since z_1 corresponds to the states x_1, so there will be two transitions separately for each letter read at z_1. It will give two possibilities of states which correspond to either z_1 or different from z_1. This process may be expressed in the following transition table for all possible states of FA_3

a,b



1



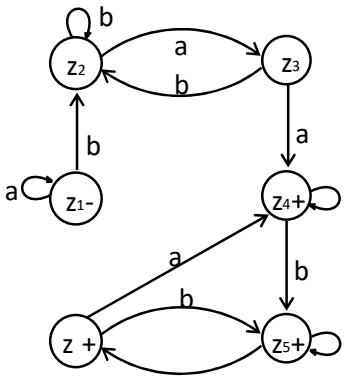
x_2+

a

Old States	New States after reading	
	a	b
z_1-≡x_1	x_1≡z_1	(x_2,y_1)≡ z_2
z_2≡(x_2,y_1)	(x_1,y_2)≡z_3	(x_2,y_1)≡ z_2
z_3≡(x_1,y_2)	(x_1,y_3)≡z_4	(x_2,y_1)≡ z_2
z_4+≡(x_1,y_3)	(x_1,y_3)≡ z_4	(x_2,y_1,y_3)≡ z_5
z_5+≡(x_2,y_1,y_3)	(x_1,y_2 ,y_3)≡ z_6	(x_2,y_1,y_3)≡ z_5
z_6+≡(x_1,y_2,y_3)	(x_1,y_3)≡ z_4	(x_2 ,y_1,y_3)≡ z_5

Hence the required FA will be as follows

a



6 b

a

Note: Another example is discussed in the next lecture.

Theory of Automata

Lecture N0. 14
Reading Material

Introduction to Computer Theory Chapter 7

Summary

Examples of Kleene's theorem part III (method 1) continued ,Kleene's theorem part III (method 2:

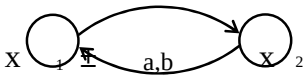
Concatenation of FAs), Examples of Kleene's theorem part III(method 2:concatenation FAs) continued, Kleene's theorem part III (method 3:closure of an FA), examples of Kleene's theorem part III(method 3:Closure of an FA) continued

MAS ALL

Example

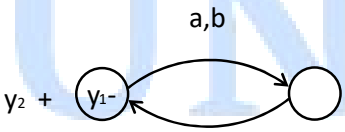
Let $r_1 = ((a+b)(a+b))^*$ and the corresponding FA_1 be

a,b



also $r_2 = (a+b)((a+b)(a+b))^*$ or $((a+b)(a+b))^*(a+b)$ and FA_2 be

ROUNDER



a,b

FA corresponding to r_1r_2 can be determined as

Old States	New States after reading a	New States after reading b
z_1- $\equiv(x_1,y_1)$	(x_2,y_2) $\equiv z_2$	$(x_2,y_2) \equiv$ z_2
z + $\equiv$(x ,y)	(x ,y) $\equiv$z	(x ,y) $\equiv$ z

Hence the required FA will be as follows

a,b



Method3: (Closure of an FA)

a,b

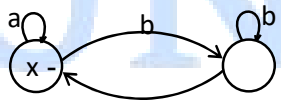
Building an FA corresponding to r^*, using the FA corresponding to r.

It is to be noted that if the given FA already accepts the language expressed by the closure of certain RE, then the given FA is the required FA. However the method, in other cases, can be developed considering the following examples

Closure of an FA, is same as concatenation of an FA with itself, except that the initial state of the required FA is a final state as well. Here the initial state of given FA, corresponds to the initial state of required FA and a non final state of the required FA as well.

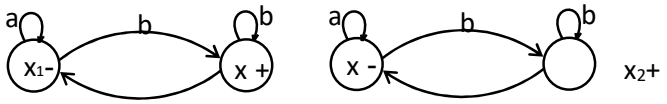
Example

Let r = $(a+b)^*b$ and the corresponding FA be



x_2 + a

then the FA corresponding to r^* may be determined as under

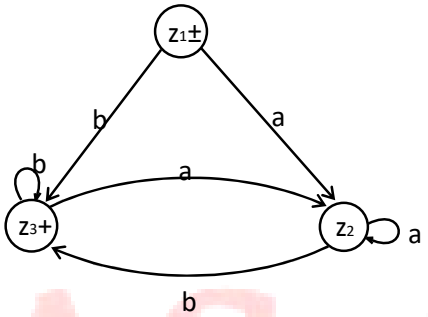


a

Old States	New States after reading	
	a	b
Final $z_1 \pm \equiv x_1$	$x_1 \equiv z_2$	$(x_2, x_1) \equiv z_3$
Non-final $z_2 \equiv x_1$	$x_1 \equiv z_2$	$(x_2, x_1) \equiv z_3$
$z + \equiv (x, x)$	$x \equiv z$	$(x, x) \equiv z$

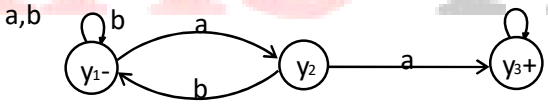
3 2 1 1 1 2 2 transition diagram may be as under

The corresponding



Example

Let r = $(a+b)^*aa(a+b)^*$ and the corresponding FA be



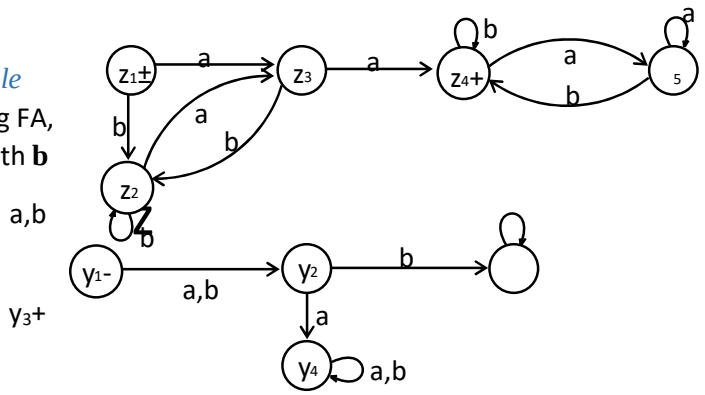
then the FA corresponding to r^* may be determined as under

Old States	New States after reading	
	a	b
Final $z_1 \pm \equiv y_1$	$y_2 \equiv z_3$	$y_1 \equiv z_2$
Non-Final $z_2 \equiv y_1$	$y_2 \equiv z_3$	$y_1 \equiv z_2$
$z_3 \equiv y_2$	$(y_3, y_1) \equiv z_4$	$y_1 \equiv z_2$
$z_4 + \equiv (y_3, y_1)$	$(y_3, y_1, y_2) \equiv z_5$	$(y_3, y_1) \equiv z_4$
$z_5 + \equiv (y_3, y_1, y_2)$	$(y_3, y_1, y_2) \equiv z_5$	$(y_3, y_1) \equiv z_4$

The corresponding transition diagram may be

z + Example

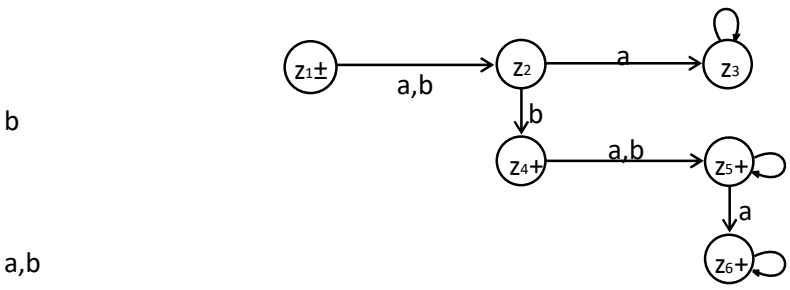
Consider the following FA, accepting the language of strings with **b** **as second letter**



then the FA corresponding to r^* may be determined as under

Old States	New States after reading	
	a	b
$z_1 \pm \equiv y_1$	$y_2 \equiv z_2$	$y_2 \equiv z_2$
$z_2 \equiv y_2$	$y_4 \equiv z_3$	$(y_3,y_1) \equiv z_4$
$z_3 \equiv y_4$	$y_4 \equiv z_3$	$y_4 \equiv z_3$
$z_4+ \equiv (y_3,y_1)$	$(y_3,y_1,y_2) \equiv z_5$	$(y_3,y_1,y_2) \equiv z_5$
$z_5+ \equiv (y_3,y_1,y_2)$	$(y_3,y_1,y_2,y_4) \equiv z_6$	$(y_3,y_1,y_2) \equiv z_5$
$z_6 \equiv (y_1,y_1,y_2,y_4)$	$(y_1,y_1,y_2,y_4) \equiv z_6$	$(y_1,y_1,y_2,y_4) \equiv z_6$

The corresponding transition diagram may be



MAS ALL ROUNDER

Reading Material

Introduction to Computer Theory Chapter 7

Summary

Examples of Kleene's theorem part III (method 3), NFA, examples, avoiding loop using NFA, example, converting FA to NFA, examples, applying an NFA on an example of maze

Note

It is to be noted that as observed in the examples discussed in previous lecture, if at the initial state of the given FA, there is either a loop or an incoming transition edge, the initial state corresponds to the final state and a nonfinal state as well, of the required FA, otherwise the initial state of given FA will only correspond to a single state of the required FA (*i.e.* the initial state which is final as well).

Nondeterministic Finite Automaton (NFA)

Definition

An NFA is a TG with a unique start state and a property of having single letter as label of transitions. An NFA is a collection of three things

Finite many states with one initial and some final states

Finite set of input letters, say, Σ = {a, b, c}

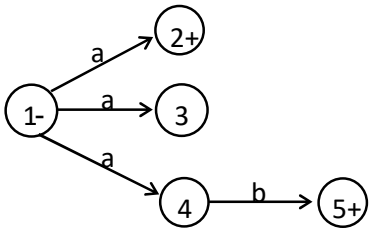
Finite set of transitions, showing where to move if a letter is input at certain state (∧ is not a valid transition), there may be more than one transition for certain letters and there may not be any transition for certain letters.

Observations

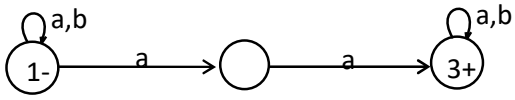
It may be observed, from the definition of NFA, that the string is supposed to be accepted, if there exists at least one successful path, otherwise rejected.

It is to be noted that an NFA can be considered to be an intermediate structure between FA and TG. The examples of NFAs can be found in the following

Example



It is to be noted that the above NFA accepts the language consisting of a and ab.



It is to be noted that the above NFA accepts the language of strings, defined over Σ = {a, b}, containing aa.

Example

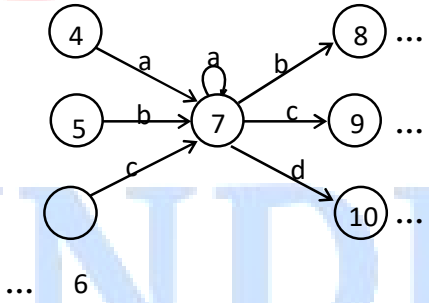
2

Note

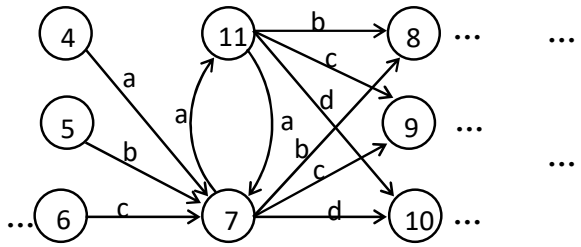
It is to be noted that NFA helps to eliminate a loop at certain state of an FA. This process is done converting the loop into a circuit. But during this process the FA remains no longer FA and is converted to a corresponding NFA, which is shown in the following example.

Consider a part of the following FA with an

Example … alphabet Σ = {a,b,c,d}



To eliminate the loop at state 7, the corresponding NFA may be as follows

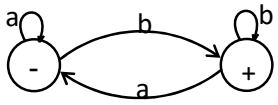


Converting an FA to an equivalent NFA

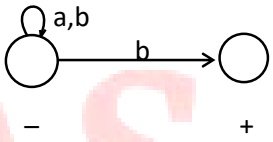
It is to be noted that according to the Kleene's theorem, if a language can be accepted by an FA, then there exists a TG accepting that language. Since, an NFA is a TG as well, therefore there exists an NFA accepting the language accepted by the given FA. In this case these FA and NFA are said to be equivalent to each others. Following are the examples of FAs to be converted to the equivalent NFAs

Example

Consider the following FA corresponding to (a+b)*b



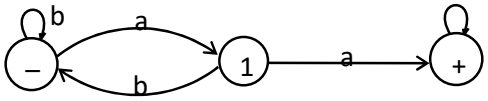
The above FA may be equivalent to the following NFA



Can the structure of above NFA be compared with the corresponding RE ?

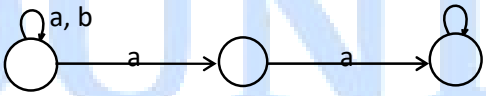
Example a,b

Consider the following



FA

The above FA may be



equivalent to the following NFA

a, b

– 1 +

Can the structure of above NFA be compared with the corresponding RE ?

Application of an NFA

There is an important application of an NFA in artificial intelligence, which is discussed in the following example of a maze

-	1	2	3
4	L	5	O
6	M	7	P
8	N	9	+

Theory of Automata

Lecture N0. 16

Reading Material

Introduction to Computer Theory Chapter 7

Summary

Applying an NFA on an example of maze, NFA with null string, examples, RE corresponding to NFA with null string (task), converting NFA to FA (method 1,2,3) examples

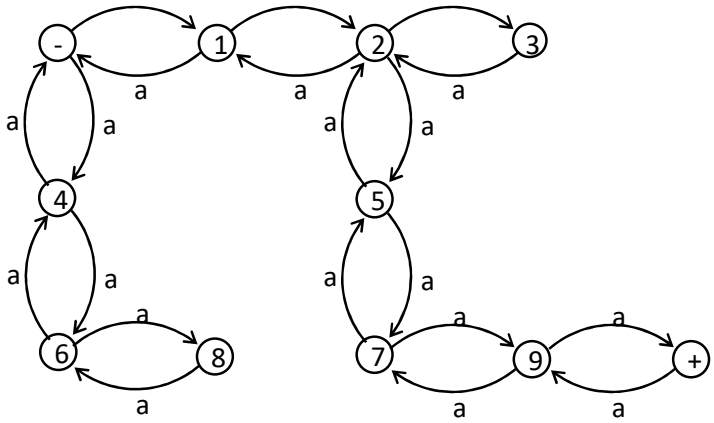
Application of an NFA

There is an important application of an NFA in artificial intelligence, which is discussed in the following example of a maze

-	1	2	3
4	L	5	O
6	M	7	P
8	N	9	+

- and + indicate the initial and final states respectively. One can move only from a box labeled by other then L, M, N, O, P to such another box. To determine the number of ways in which one can start from the initial state and end in the final state, the following NFA using only single letter a, can help in this regard

a a a



It can be observed that the shortest path which leads from the initial state and ends in the final state, consists of six steps i.e. the shortest string accepted by this machine is aaaaaa. The next larger accepted string is aaaaaaaa. Thus if this NFA is considered to be a TG then the corresponding regular expression may be written as aaaaaa(aa)*

Which shows that there are infinite many required ways

Note

It is to be noted that every FA can be considered to be an NFA as well , but the converse may not true.

It may also be noted that every NFA can be considered to be a TG as well, but the converse may not true. It may be observed that if the transition of null string is also allowed at any state of an NFA then what will be the behavior in the new structure. This structure is defined in the following

NFA with Null String

Definition

If in an NFA, $\wedge$ is allowed to be a label of an edge then the NFA is called NFA with $\wedge$ (NFA-$\wedge$).

An NFA-$\wedge$ is a collection of three things

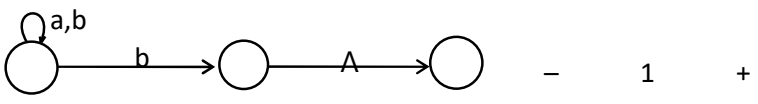
Finite many states with one initial and some final states. Finite
set of input letters, say, Σ = {a, b, c}.

Finite set of transitions, showing where to move if a letter is input at certain state.

There may be more than one transitions for certain letter and there may not be any transition for a certain letter. The transition of $\wedge$ is also allowed at any state.

Example

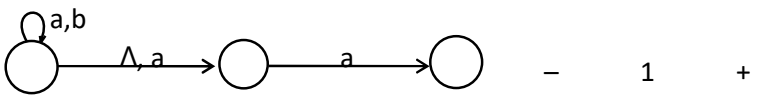
Consider the following NFA with Null string



The above NFA with Null string accepts the language of strings, defined over Σ = {a, b}, **ending in b**.

Example

Consider the following NFA with Null string



The above NFA with Null string accepts the language of strings, defined over Σ = {a, b}, **ending in a**.

Note

It is to be noted that every FA may be considered to be an NFA-∧ as well, but the converse may not true. Similarly every NFA-∧ may be considered to be a TG as well, but the converse may not true.

NFA to FA

Two methods are discussed in this regard.

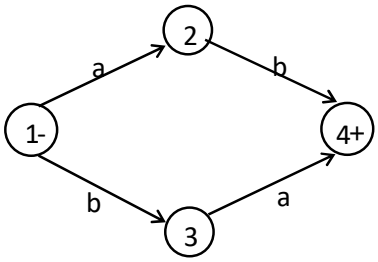
Method 1: Since an NFA can be considered to be a TG as well, so a RE corresponding to the given NFA can be determined (using Kleene's theorem). Again using the methods discussed in the proof of Kleene's theorem, an FA can be built corresponding to that RE. Hence for a given NFA, an FA can be built equivalent to the NFA. Examples have, indirectly, been discussed earlier.

Method 2: Since in an NFA, there may be more than one transition for a certain letter and there may not be any transition for certain letter, so starting from the initial state corresponding to the initial state of given NFA, the transition diagram of the corresponding FA, can be built introducing an empty state for a letter having no transition at certain state and a state corresponding to the combination of states, for a letter having more than one transitions. Following are the examples

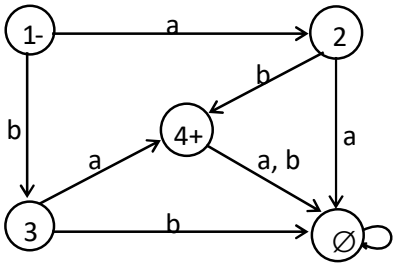
Example

Consider the following NFA

Using the method discussed earlier, the above NFA may be equivalent to the following FA

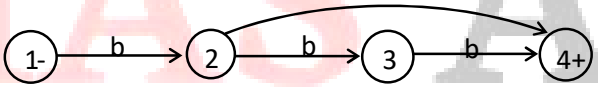


a, b

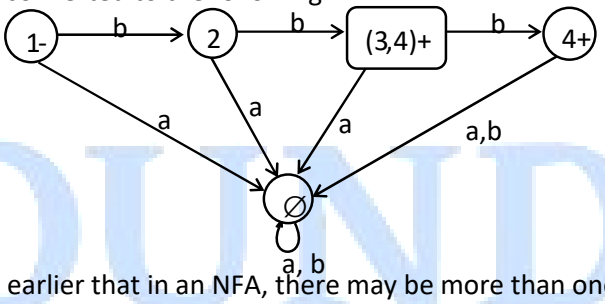


Example

A simple NFA that accepts the language of strings defined over Σ = {a,b}, **consists of bb and bbb** b



The above NFA can be converted to the following FA



Method 3 As discussed earlier that in an NFA, there may be more than one transition for a certain letter and there may not be any transition for certain letter, so starting from the initial state corresponding to the initial state of given NFA, the transition table along with new labels of states, of the corresponding FA, can be built introducing an empty state for a letter having no transition at certain state and a state corresponding to the combination of states, for a letter having more than one transitions. Further examples are discussed in the next lecture.

Theory of Automata

Lecture N0. 17

Reading Material

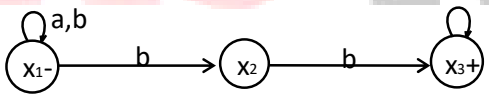
Introduction to Computer Theory Chapter 7

Summary converting NFA to FA (method 3), example, NFA and Kleene's theorem method 1, examples, NFA and Kleene's theorem method 2 , NFA corresponding to union of FAs, example

Method 3: As discussed earlier that in an NFA, there may be more than one transition for a certain letter and there may not be any transition for certain letter, so starting from the initial state corresponding to the initial state of given NFA, the transition table along with new labels of states, of the corresponding FA, can be built introducing an empty state for a letter having no transition at certain state and a state corresponding to the combination of states, for a letter having more than one transitions. Following are the examples

Example

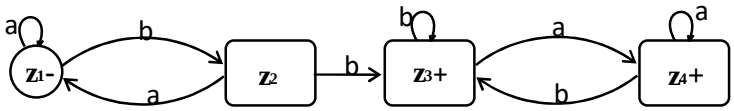
Consider the following NFA which accepts the language of strings **containing bb** a,b



Using the method discussed earlier, the transition table corresponding to the required FA may be constructed as

Old States	New States after reading	
	a	b
$z_1 - \equiv x_1$	$x_1 \equiv z_1$	$(x_1,x_2) \equiv z_2$
$z_2 \equiv (x_1,x_2)$	$(x_1, \varnothing) \equiv x_1 \equiv z_1$	$(x_1,x_2,x_3) \equiv z_3$
$z_3 + \equiv (x_1,x_2,x_3)$	$(x_1,x_3) \equiv z_4$	$(x_1,x_2,x_3) \equiv z_3$
$z_4 + \equiv (x_1,x_3)$	$(x_1,x_3) \equiv z_4$	$(x_1,x_2,x_3) \equiv z_3$

The corresponding transition diagram follows as



NFA and Kleene's Theorem

It has been discussed that, by Kleene's theorem part III, there exists an FA corresponding to a given RE. If the given RE is as simple as r = aa+bbb or r = a(a+b)*, the corresponding FAs can easily be constructed. However, for a complicated RE, the RE can be decomposed into simple REs corresponding to which the FAs can easily be constructed and hence, using the method, constructing the FAs corresponding to sum, concatenation and closure of FAs, the required FA can also be constructed. It is to be noted that NFAs also help in proving Kleene's theorem part III, as well. Two methods are discussed in the following.

NFA and Kleene's Theorem Method 1:

The method is discussed considering the following example.

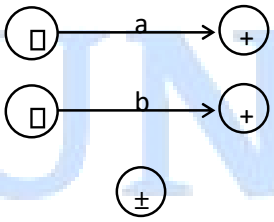
MAS ALL ROUNDER

Example

To construct the FAs for the languages L1 = {a}, L2 = {b} and L3 = {∧}

Step 1: Build NFA_1, NFA_2 and NFA_3 corresponding to L_1, L_2 and L_3 , respectively as shown in the following diagram

NFA_1



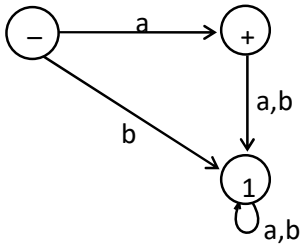
NFA_2

NFA_3

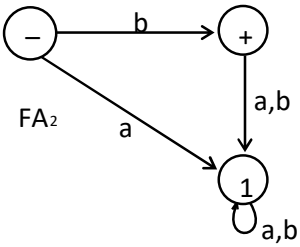
Step 2:

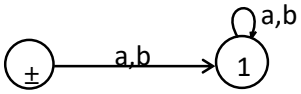
As discussed earlier for every NFA there is an FA equivalent to it, hence there must be FAs for the above mentioned NFAs as well. The corresponding FAs can be considered as follows

FA1



FA2





FA3

NFA and Kleene's Theorem method 2

It may be observed that if an NFA can be built corresponding to union, concatenation and closure of FAs corresponding to the REs, then converting the NFA, thus, obtained into an equivalent FA, this FA will correspond to the given RE.

Followings are the procedures showing how to obtain NFAs equivalent to union, concatenation and closure of

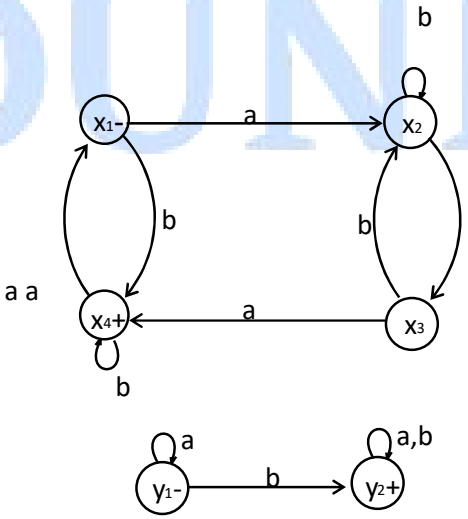
FAs

NFA corresponding to Union of FAs Method

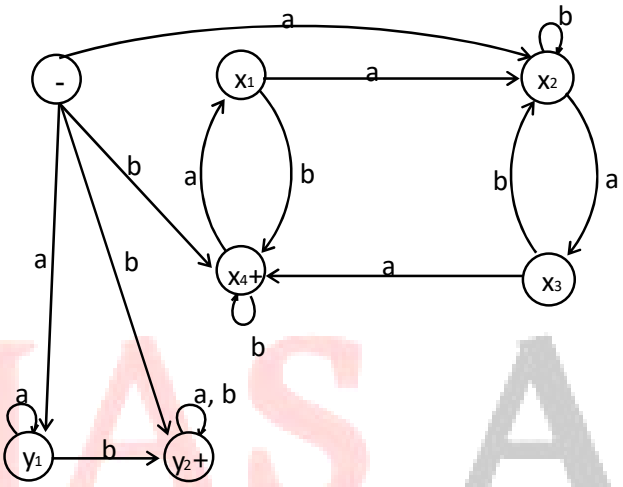
Introduce a new start state and connect it with the states originally connected with the old start state with the same transitions as the old start state, then remove the –ve sign of old start state. This creates non-determinism and hence results in an NFA.

Example

FA1



FA2



MAS ALL

NFA equivalent to $FA_1 \cup FA2$

ROUNDER

Theory of Automata

Lecture N0. 18

Reading Material

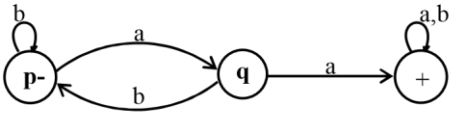
Introduction to Computer Theory Chapter 7

Summary

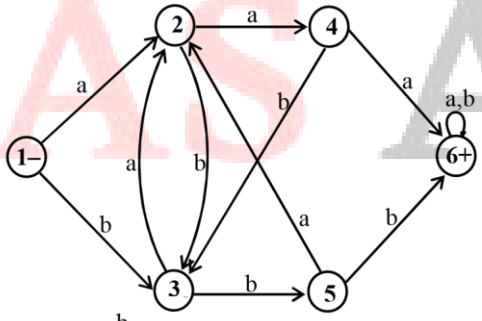
NFA corresponding to union of FAs, example, NFA corresponding to concatenation of FAs, examples, NFA corresponding to closure of an FA, example

Example

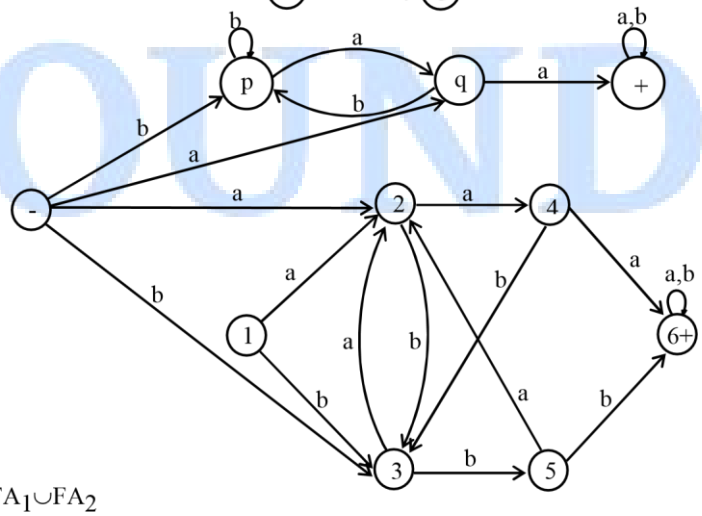
FA_1



FA_2



NFA equivalent to $FA_1 \cup FA_2$



NFA corresponding to Concatenation of FAs

Method

Introduce additional transitions for each letter connecting each final state of the first FA with the states of second FA that are connected with the initial state of second FA corresponding to each letter of the alphabet. Remove the +ve sign of each of final states of first FA and –ve sign of the initial state of second

FA. It will create non-determinism at final states of first FA and hence NFA, thus obtained, will be the required NFA.

Note

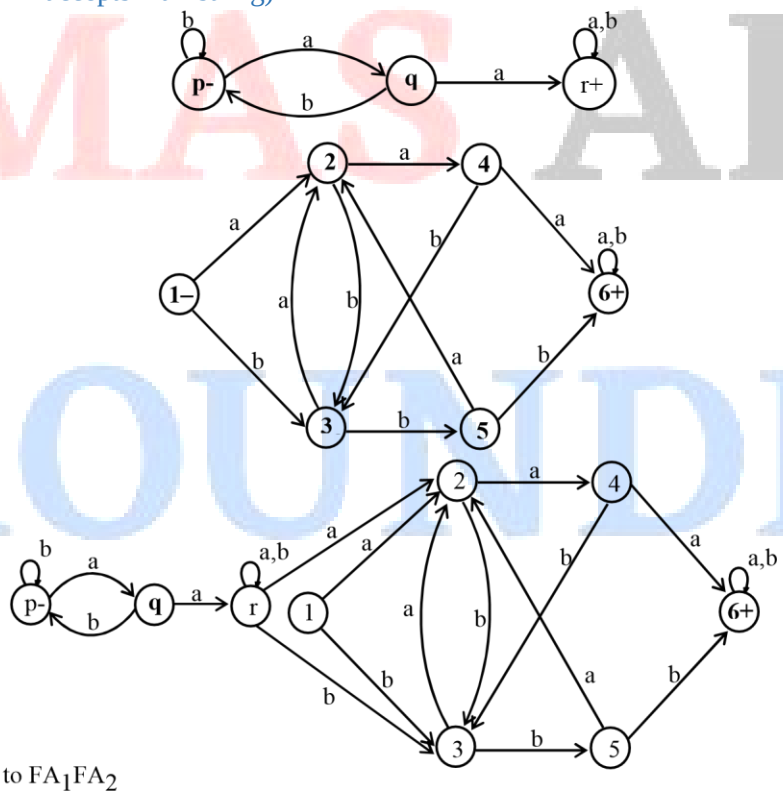
It may be noted that if first FA accepts the Null string then every string accepted by second FA must be accepted by the concatenation of FAs as well. This situation will automatically be accommodated using the method discussed earlier. However if the second FA accepts Null string, then every string accepted by first FA must be accepted by the required FA as well. This target can be achieved as, while introducing new transitions at final states of first FA the +ve sign of these states will not be removed.

Lastly if both FAs accepts the Null string, then the Null string must be accepted by the required FA. This situation will automatically be accommodated as the second FA accepts the Null string and hence the +ve signs of final states of first FA will not be removed.

Example (No FA accepts Null string)

FA_1

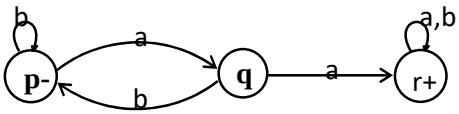
FA_2



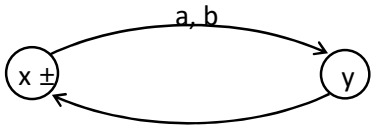
NFA equivalent to FA_1FA_2

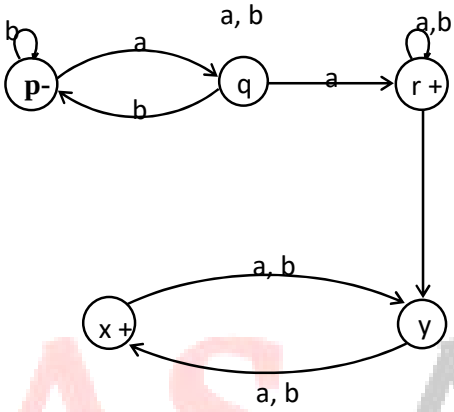
Example (FA_2 accepts Null string)

FA1



FA2





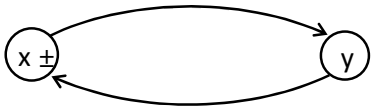
NFA equivalent to FA_1FA2

Example (Both FAs accept Null string)

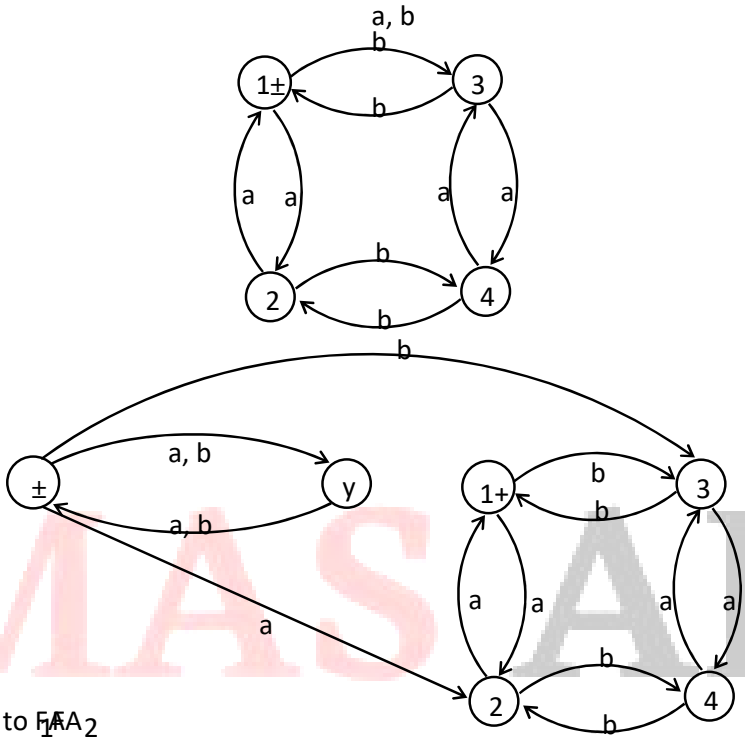
a, b

MAS ALL ROUNDER

FA1



FA2



NFA equivalent to FA1+FA2

NFA corresponding to the Closure of an FA

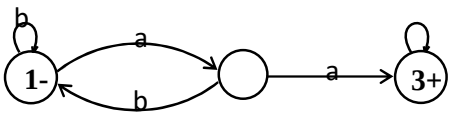
Apparently, it seems that since closure of an FA accepts the Null string, so the required NFA may be obtained considering the initial state of given FA to be final as well, but this may allow the unwanted string to be accepted as well. For example, an FA, with two states, accepting the language of strings, defined over. Σ = {a, b}, **ending in a**, will accept all unwanted strings, if the initial state is supposed to be final as well.

Method

Thus, to accommodate this situation, introduce an initial state which should be final as well (so that the Null string is accepted) and connect it with the states originally connected with the old start state with the same transitions as the old start state, then remove the –ve sign of old start state. Introduce new transitions, for each letter, at each of the final states (including new final state) with those connected with the old start state.This creates non-determinism and hence results in the required NFA.

Example

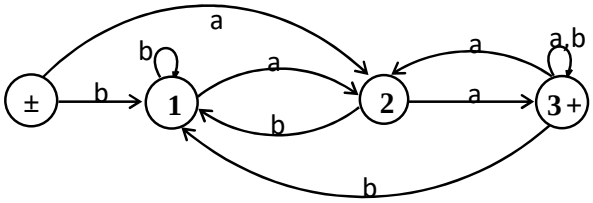
Consider the following FA



2

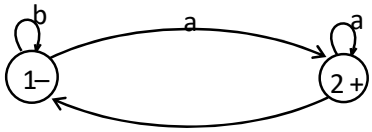
It may be observed that the FA* accepts only the additional string which is the Null string.

Considering the state 1 to be final as well, will allow the unwanted strings be accepted as well. Hence the required NFA is constructed introducing the new initial state, shown below.



Example

Consider the following FA

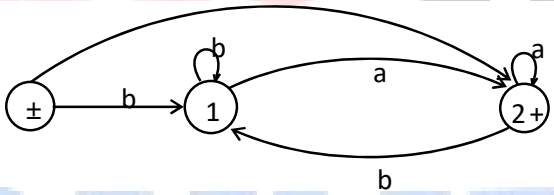


b

It may be observed that the FA* accepts only the additional string which is the Null string

As observed in the previous example the required NFA can be constructed only if the new initial state is introduced as shown below.

a



MAS ALL ROUNDER

Theory of Automata

Lecture N0. 19

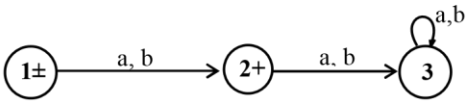
Reading Material

Introduction to Computer Theory Chapter 7

Summary

NFA corresponding to Closure of FA, Examples, Memory required to recognize a language, Example, Distinguishing one string from another, Example, Theorem, Proof

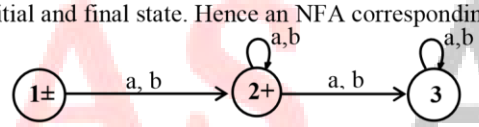
Example

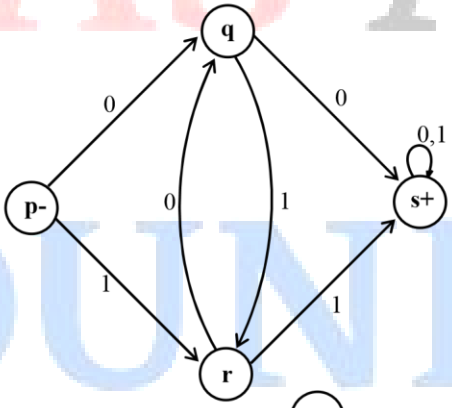


Consider the following FA

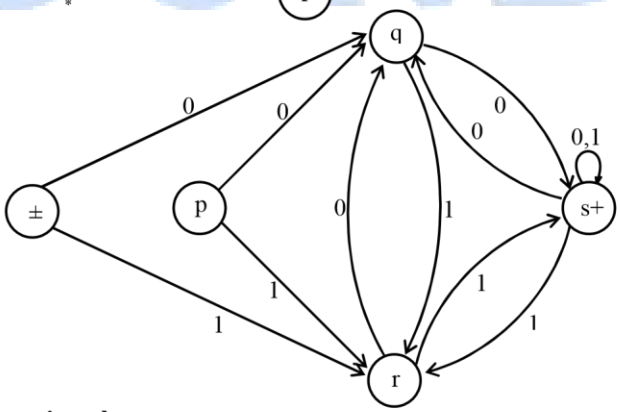
It can be observed that FA* not only accepts the Null string but every other string as well.
Here we don't need separate initial and final state. Hence an NFA corresponding to FA may be

Example





Consider the following FA



The NFA corresponding to FA may be as follows

Memory required to recognize a language

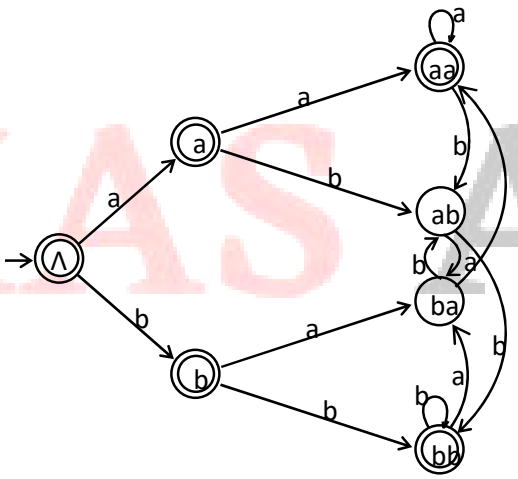
MAS ALL ROUNDER

Memory required to recognize a language means to look at the machine which can recognize a language. As an FA can be considered to be a machine which is simple model of computation and every regular language is associated with certain FA, so to recognize a language there is a restriction that there is a single pass from left to right for any string to decide whether it belongs to certain language ? This helps to remember the information about the initial part of the string read so far.

By this process the input string is examined and the string is decided either to be in a certain language or not.

Consider L = {w ∈ {a,b}*: w neither ends in **ab** nor in **ba**}. *i.e.* L is the language of strings, defined over Σ = {a,b}, consisting of Λ, a, b and strings ending in aa or bb. L may be accepted by the following FA

a



MAS ALL ROUNDER

As seen in the above FA, seven states are required to recognize the language L, while on the other hand it is very hard to recognize the language PALINDROME.

As seen in the above example of FA, seven states are required to recognize that language. Now consider another language L_3 of strings of length three or more, defined over Σ = {a,b}, **and the third letter from the right is a.** As discussed by Martin, there is a straight forward method to build an FA recognizing L_3 *i.e.* a distinct state for every possible substring of length less then or equal to 3. It is obvious that for each length *i*, *i*=0,1,2,3, of substring, the number of states are 2^i and thus total number of states required to recognize the language L_3 are $2^0+2^1+2^2+2^3 = 2^{3+1}-1=15$ (using $2^0+2^1+2^2+\ldots+ 2^n= 2^{n+1}-1$)

Remark: Let L_{20} be the language of strings of length 20 or more, defined over Σ = {a,b}, **and the 20th letter from the right is 1,** then following the previous method, number of states for the corresponding FA is

2^{20+1}-1=2,097,151.

However, it may be noted that any portion of memory of a computer that can accommodate 21 bits can be in 2^{21} possible states *i.e.* 2^{21} possible choices for the informational content.

Distinguishable strings and Indistinguishable strings

Two strings x and y, belonging to Σ^*, are said to be **distinguishable** w.r.t a language $L \subseteq \Sigma^*$ if there exists a string z belonging to Σ^* s.t. $xz \in L$ but $yz \notin L$ or $xz \notin L$ but $yz \in L$.

Two strings x and y, belonging to Σ^*, are said to be **indistinguishable** with respect to a language $L \subseteq \Sigma^*$ if for every string z belonging to Σ^*, either both xz or yz $\in$ L or both don't belong to L.

Example

Let L be the language of strings, defined over $\Sigma = \{0,1\}$, **ending in 01.**

The strings 110 and 010011 are **distinguishable** w.r.t L, as there exists 1 belonging to Σ^* s.t. 1101 belongs to L but 0100111 doesn't belong to L.

But 111 and 010011 are **indistinguishable**, for 1 belonging to Σ^* s.t. both 1111 and 010011 don't belong to L *i.e.* for every z belonging to Σ^*, either both 111z and 01001z belong to L, or both don't belong to L.

Theorem

Statement

If L is a language over an alphabet $\sum$ and for integer n there are n strings from $\sum^*$, any two of which are distinguishable w.r.t. language L, then any FA recognizes L must have at least n states. (Note: There may not exist any FA which recognizes the given language.)

Proof

Let S be set of strings, any two of which are distinguishable w.r.t. language L. Let F_1 be the FA which recognizes the language L. To prove the theorem, it is sufficient to show that any two strings under F_1 must be ended in different states *i.e.* corresponding to each string x belonging to S, F_1 ends in distinct states. Thus if S has n strings then it is to be shown that F_1 has at least n states.

Let x and y be any two strings from S. By supposition any two strings of S are distinguishable w.r.t. L, so there exists a string z belonging to $\sum^*$such that only one of xz and yz belongs to L *i.e.*F_1 ends in a final state either for xz or yz which shows that F_1 ends in distinct states for xz and yz.

Let F_1 be ended in same state for both the strings x and y, which shows that F_1ends in same state for both xz and yz, a contradiction as x and y being distinguishable implies xz and yz are ended at distinct states of F_1. Hence F_1 does not end in a same state for both strings x and y, which shows that each pair of strings belonging to S ends in different states. Hence F_1 must contain at least n states.

Theory of Automata

Lecture N0. 20

Reading Material

Introduction to Computer Theory Chapter 8

Summary

Example of previous Theorem, Finite Automaton with output, Moore machine, Examples

Example

Let L_{20}={w ∈ {0,1}*: |w| ≥ 20 and the 20th letter of w, from right is, 1}. Let S be the set of all strings of length 20, defined over Σ, any two of which are distinguishable *w.r.t.* L_{20}. Obviously the number of strings belonging to S, is 2^{20}. Let x and y be any two distinct strings *i.e.* they differ in ith letter, i=1,2,3,…20, from left. For i=1, they differ by first letter from left.

Then by definition of L_{20}, one is in L_{20} while other is not as shown below

0	.	…	.

1	.	…	.

So they are distinct *w.r.t.* L_{20} for z = Λ *i.e.* one of xz and yz belongs to L_{20}.

Similarly if i=2 they differ by 2nd letter from left and are again distinguishable and hence for z belonging to Σ*,

|z|=1, either xz or yz belongs to L_{20} because in this case the 20th letter from the right of xz and yz is exactly the 2nd letter from left of x and y as shown below

.	0	…	.	z

.	1	…	.	z

Hence x and y will be distinguishable *w.r.t.* L_{20} for i=2, as well. Continuing the process it can be shown that any pair of strings x and y belonging to S, will be distinguishable *w.r.t.* L_{20}. Since S contains 2^{20} strings, any two of which are distinguishable *w.r.t.* L_{20}, so using the theorem any FA accepting L_{20} must have at least 2^{20} states.

Note

It may be observed from the above example that using Martin's method, there exists an FA having

2^{20+1}-1=2,097,151 states. This indicates the memory required to recognize L_{20} will be the memory of a computer that can accommodate 21-bits *i.e.*the computer can be in 2^{21} possible states.

Finite Automaton with output

Finite automaton discussed so far, is just associated with the RE or the language.

There is a question whether does there exist an FA which generates an output string corresponding to each input string ? The answer is yes. Such machines are called machines with output. There are two types of machines with output. Moore machine and Mealy machine

Moore machine

A Moore machine consists of the following

A finite set of states q_0, q_1, q_2, … where q_0 is the initial state.

An alphabet of letters Σ = {a,b,c,…} from which the input strings are formed.

An alphabet Γ={x,y,z,…} of output characters from which output strings are generated.

A transition table that shows for each state and each input letter what state is entered the next. An output table that shows what character is printed by each state as it is entered.

Note

It is to be noted that since in Moore machine no state is designated to be a final state, so there is no question of accepting any language by Moore machine. However in some cases the relation between an input string and the corresponding output string may be identified by the Moore machine. Moreover, the state to be initial is not important as if the machine is used several times and is restarted after some time, the machine will be started from the state where it was left off. Following are the examples

Example

Consider the following Moore machine having the states q_0, q_1, q_2, q_3 where q_0 is the start state and Σ = {a,b}, Γ={0,1}

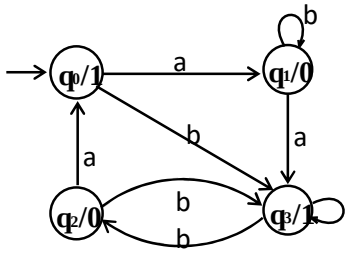
the transition table follows as

Old States	**New States after reading**		Characters to be printed
	a	**b**	
q_0-	**q_1**	**q_3**	1
q_1	**q_3**	**q_1**	0
q_2	**q_0**	**q_3**	0

q_3	q_3	q_2	1

the transition diagram corresponding to the previous transition table may be

a



It is to be noted that the states are labeled along with the characters to be printed. Running the string abbabbba over the above machine, the corresponding output string will be 100010101, which can be determined by the following table as well

Input		a	b	b	a	b	b	b	a
State	q_0	q_1	q_1	q_1	q_3	q_2	q_3	q_2	q_0
output	1	0	0	0	1	0	1	0	1

It may be noted that the length of output string is l more than that of input string as the initial state prints out the extra character 1, before the input string is read.

Theory of Automata

Lecture N0. 21

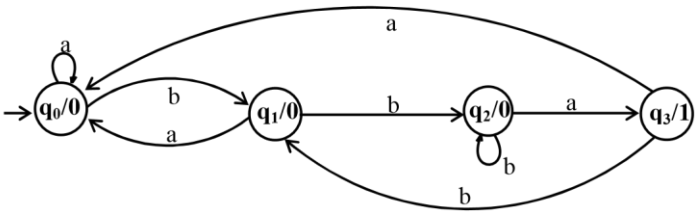
Reading Material

Introduction to Computer Theory Chapter 8

Summary

Example of Moore machine, Mealy machine, Examples, complementing machine, Incrementing machine.

Example



To identify the relation between the input strings and the corresponding output strings in the following Moore machine,

if the string bbbabaabbaa is run, the output string will be 000010000010, as shown below

Input		b	b	b	a	b	a	a	b	b	a	a
State	q_0	q_1	q_2	q_2	q_3	q_1	q_0	q_0	q_1	q_2	q_3	q_0
output	0	0	0	0	1	0	0	0	0	0	1	0

MAS ALL ROUNDER

It can be observed from the given Moore machine that q_3 is the only state which prints out the character 1 which shows that the moment the state q_3 is entered, the machine will print out 1. To enter the state q_3, starting from q_0 the string must contain bba. It can also be observed that to enter the state q_3 once more the string must contain another substirng bba. In general the input string will visit the state q_3 as many times as the number of substring bba occurs in the input string. Thus the number of 1's in an output string will be same as the number of substring bba occurs in the corresponding input string.

Mealy machine

A Mealy machine consists of the following

A finite set of states q_0, q_1, q_2, … where q_0 is the initial state.

An alphabet of letters Σ = {a,b,c,…} from which the input strings are formed.

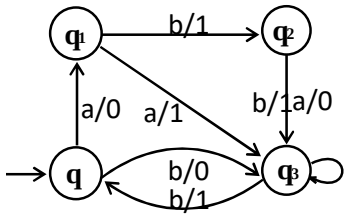
An alphabet Γ={x,y,z,…} of output characters from which output strings are generated.

A pictorial representation with states and directed edges labeled by an input letter along with an output character. The directed edges also show how to go from one state to another corresponding to every possible input letter.

(It is not possible to give transition table in this case.)

Note

It is to be noted that since, similar to Moore machine, in Mealy machine no state is designated to be a final state, so there is no question of accepting any language by Mealy machine. However in some cases the relation between an input string and the corresponding output string may be identified by the Mealy machine. Moreover, the state to be initial is not important as if the machine is used several times and is restarted after some time, the machine will be started from the state where it was left



off. Following are the examples

0 a/1

Example

Consider the Mealy machine shown aside, having the states q_0, q_1, q_2, q_3, where q_0 is the start state and

Σ = {a,b},

Γ={0,1}

Running the string abbabbba over the above machine, the corresponding output string will be 11011010, which can be determined by the following table as well

Input		a	b	b	a	b	b	b	a
States	q_0	q_1	q_2	q_3	q_3	q_0	q_3	q_0	q_1
output		0	1	1	1	1	0	1	0

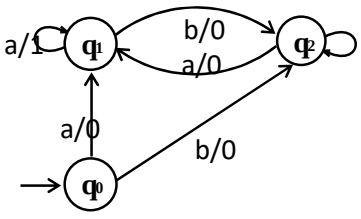
It may be noted that in Mealy machine, the length of output string is equal to that of input string.

Example

Consider the following Mealy machine having the states q_0, q_1, q_2, where q_0 is the start state and

Σ = {a,b},

Γ={0,1} b/1



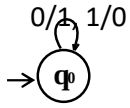
It is observed that in the above Mealy machine, if in the output string the nth character is 1, it shows that the nth letter in the input string is the second in the pair of double letter. For babaababba as input string the machine will print 0000100010.

Example

Consider the following Mealy machine having the only state q_0 as the start state and

$\Sigma = \{0,1\}$,

$\Gamma = \{0,1\}$



If 0011010 is run on this machine then the corresponding output string will be 1100101.
This machine is called **Complementing machine.**

Constructing the incrementing machine

In the previous example of complementing machine, it has been observed that the input string and the corresponding output string are 1's complement of each other. There is a question whether the Mealy machine can be constructed, so that the output string is increased, in magnitude, by 1 than the corresponding input string?

The answer is yes.

This machine is called the incrementing machine. Following is how to construct the incrementing machine. Before the incrementing machine is constructed, consider how 1 is added to a binary number.

Since, if two numbers are added, the addition is performed from right to left, so while increasing the binary number by 1, the string (binary number) must be read by the corresponding Mealy machine from right to left, and hence the output string (binary number) will also be generated from right to left.

Consider the following additions

a) 100101110 + 1 = 100101111

b) 1001100111 + 1 = 1001101000

It may be observed from the above that

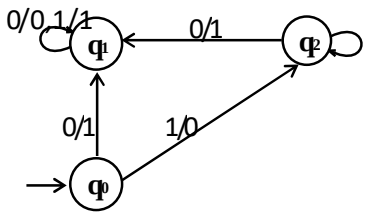
If the right most bit of binary number, to be incremented, is 0, the output binary number can be obtained by converting the right most bit to 1 and remaining bits unchanged.

If the right most bit of binary number is 1 then the output can be obtained, converting that 1 along with all its concatenated 1's to 0's, then converting the next 0 to 1 and remaining bits unchanged. The observations (a) and (b) help to construct the following Incrementing (Mealy) machine. The Mealy machine have the states q_0, q_1, q_2, where q_0 is the start state and

Σ = {0,1},

Γ={0,1}

1/0



It may be observed that, in the incrementing machine, if 0 is read at initial state $\mathbf{q_0}$, that 0 is converted to 1 and a no change state $\mathbf{q_1}$ (no carry state) is entered where all 0's and all 1's remain unchanged. If 1 is read at initial state, that 1 is converted to 0 and the state q_2**(**owe carry state**)** is entered, where all 1's are converted to 0's and at that state if 0 is read that 0 is converted to 1 and the machine goes to no change state.

If the strings 100101110 and 1001100111 are run over this machine, the corresponding output strings will be 100101111 and 1001101000 respectively.

Note

It is to be noted that if the string 111111 is run over the incrementing machine, the machine will print out 000000, which is not increased in magnitude by 1. Such a situation is called an overflow situation, as the length of output string will be same as that of input string.

It may also be noted that there exists another incrementing machine with two states.

MAS ALL ROUNDER

Theory of Automata

Lecture N0. 22

Reading Material

Introduction to Computer Theory Chapter 8

Summary

Applications of complementing and incrementing machines, Equivalent machines, Moore equivalent to Mealy, proof, example, Mealy equivalent to Moore, proof, example

Applications of Incrementing and Complementing machines

1's complementing and incrementing machines which are basically Mealy machines are very much helpful in computing.

The incrementing machine helps in building a machine that can perform the addition of binary numbers. Using the complementing machine along with incrementing machine, one can build a machine that can perform the subtraction of binary numbers, as shown in the following method

Subtracting a binary number from another

Method

To subtract a binary number b from a binary number a

Add 1's complement of b to a (ignoring the overflow, if any)

Increase the result, in magnitude, by 1 (use the incrementing machine). Ignoring the overflow if any.

Note: If there is no overflow in (1). Take 1's complement once again in (2), instead. This situation occurs when b is greater than a, in magnitude. Following is an example of subtraction of binary numbers

Example

To subtract the binary number 101 from the binary number 1110, let

a = 1110 and b = 101 = 0101.

(Here the number of digits of b are equated with that of a)

Adding 1's complement (1010) of b to a.

1110

+1010

11000 which gives 1000 (ignoring the overflow)

Using the incrementing machine, increase the above result 1000, in magnitude, by 1

1000

+1

1001 which is the same as obtained by ordinary subtraction.

Note

It may be noted that the above method of subtraction of binary numbers may be applied to subtraction of decimal numbers with the change that 9's complement of b will be added to a, instead in step (1).

Equivalent machines

Two machines are said to be **equivalent** if they print the same output string when the same input string is run on them. Remark:

Two Moore machines may be equivalent. Similarly two Mealy machines may also be equivalent, but a Moore machine can't be equivalent to any Mealy machine. However, ignoring the extra character printed by the Moore machine, there exists a Mealy machine which is equivalent to the Moore machine.

Theorem

Statement

For every Moore machine there is a Mealy machine that is equivalent to it (ignoring the extra character printed by the Moore machine).

MAS ALL ROUNDER

Proof:

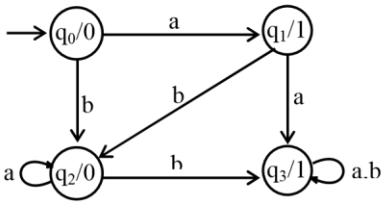
Let M be a Moore machine, then shifting the output characters corresponding to each state to the labels of corresponding incoming transitions, machine thus obtained will be a Mealy machine equivalent to M.

Note

It may be noted that while converting a Moore machine into an equivalent Mealy machine, the output character of a state will be ignored if there is no incoming transition at that state. A loop at a state is also supposed to be an incoming transition.

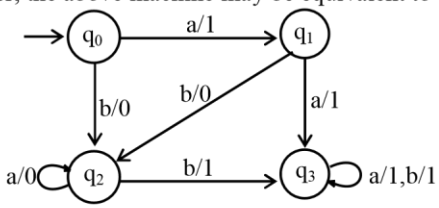
Following is the example of converting a Moore machine into an equivalent Mealy machine

Example



Consider the following Moore machine

Using the method described earlier, the above machine may be equivalent to the following Mealy machine



Running the string abbabbba on both the machines, the output string can be determined by the following table

Input		a	b	b	a	b	b	b	a
States	q_0	q_1	q_2	q_3	q_3	q_3	q_3	q_3	q_3
Moore	0	1	0	1	1	1	1	1	1
Mealy		1	0	1	1	1	1	1	1

Theorem

Statement

For every Mealy machine there is a Moore machine that is equivalent to it (ignoring the extra character printed the Moore machine).

Proof

Let M be a Mealy machine. At each state there are two possibilities for incoming transitions
The incoming transitions have the same output character.

The incoming transitions have different output characters.

If all the transitions have same output characters, then shift that character to the corresponding state. If all the transitions have different output characters, then the state will be converted to as many states as the number of different output characters for these transitions, which shows that if

this happens at state q_i then q_i will be converted to q_i^1 and q_i^2 *i.e.* if at q_i there are the transitions with two output characters then q_i^1 for one character and q_i^2 for other character.

Shift the output characters of the transitions to the corresponding new states q_i^1 and q_i^2. Moreover, these new states q_i^1 and q_i^2 should behave like q_i as well. Continuing the process, the machine thus obtained, will be a Moore machine equivalent to Mealy machine M.

Following is a note

Note

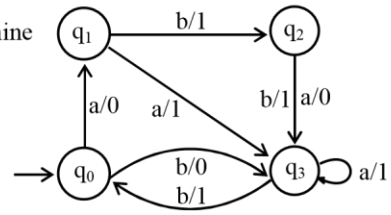
It may be noted that if there is no incoming transition at certain state then any of the output characters may be associated with that state.

It may also be noted that if the initial state is converted into more than one new states then only one of these new states will be considered to be the initial state. Following is an example

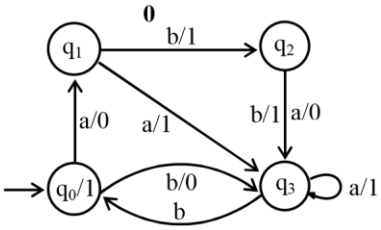
MAS ALL ROUNDER

Example

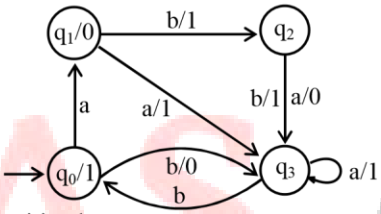
Consider the following Mealy machine



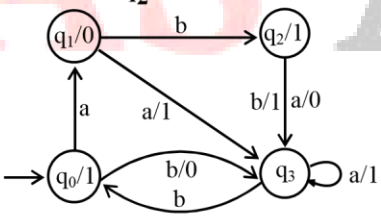
Shifting the



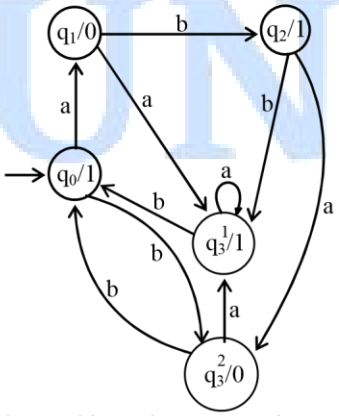
Shifting the output character 0 of transition a to $\mathbf{q_1}$



Shifting the output character 1 of transition b to $\mathbf{q_2}$



Splitting $\mathbf{q_3}$ into $\mathbf{q^1_3}$ and $\mathbf{q^2_3}$



Running the string abbabbba on both the machines, the output strings can be determined by the following table

Input		a	b	b	a	b	b	b	a
States	$\mathbf{q_0}$	$\mathbf{q_1}$	$\mathbf{q_2}$	$\mathbf{q_3}$	$\mathbf{q_3}$	$\mathbf{q_0}$	$\mathbf{q_3}$	$\mathbf{q_0}$	$\mathbf{q_1}$
Mealy		0	1	1	1	1	0	1	0
Moore	1	0	1	1	1	1	0	1	0

output character 1 of transition b to **q** Theory of Automata

MAS ALL

ROUNDER

FOR ANY VU STUDY RELATED PROBLEM
CONTACT US: 03024731376

All vu data (midterm final term+ assignments +quiz + GDB +) and also WAQAR + MOAZ +TEAM HADI + VU TOPPER RM AND ALL OTHER STUDENTS FILES VISIT

https://masallrounder.blogspot.com/

MAS ALL ROUNDER